M01 — Fundamentos de ASP.NET Core MVC

Cuaderno de prácticas

CURSO: Curso de Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: PRÁCTICAS

AUTOR: JAIME GALLO

Índice del módulo

- **P1.1. Primera aplicación MVC y recorrido de una petición HTTP**
- **P1.2. El flujo Modelo–Vista–Controlador**
- **P1.3. Estructura y convenciones del proyecto**
- **P1.4. Controladores, acciones, parámetros y rutas**
- **P1.5. Vistas tipadas, layout, parciales y navegación**

P1.1. Primera aplicación MVC y recorrido de una petición HTTP

Objetivo y preparación

Objetivo. Crear una aplicación MVC a partir de la plantilla de .NET 8, ejecutar el proyecto inicial suministrado en el curso, observar una petición real y distinguir los cambios en las cabeceras HTTP de los cambios en el HTML generado por una vista Razor.

Teoría relacionada. Punto 1.1, especialmente flujo HTTP, diferencia MVC/API, papel de Kestrel, `Program.cs`, acción MVC, vista Razor y cabeceras HTTP.

Conocimientos previos. C# básico, uso de una terminal, HTTP GET y experiencias anteriores con Web API. No se requiere conocer Razor ni su sintaxis en profundidad.

Producto de esta práctica. Una aplicación MVC que responde en `http://localhost:5091/`, cuyo estado inicial ya está dispuesto en el repositorio y cuyos dos pequeños cambios pueden activarse de forma independiente.

Herramientas y dependencias

Se necesita .NET SDK 8, navegador, editor C# (preferentemente Visual Studio con soporte ASP.NET Core) y Git. La práctica se describe con comandos de la CLI de .NET, equivalentes al botón de ejecución de Visual Studio. Para inspeccionar HTTP usaremos la pestaña *Red/Network* del navegador; como comprobación alternativa puede usarse `curl.exe` en Windows o `curl` en Linux/macOS.

Comprobar el SDK:

```
dotnet --list-sdks
```

Resultado esperado: al menos una entrada cuya versión comience por `8.0.`. Una versión posterior no sustituye necesariamente al SDK 8 en la ejecución de esta práctica, cuyo proyecto apunta explícitamente a `net8.0`.

Obtener el estado inicial exacto

En una carpeta de trabajo elegida por el alumno:

```
git clone --branch desarrollo/m01-mvc https://github.com/jaimecopilot/CURSO-DESARROLLO-WEB-NET8.git
cd CURSO-DESARROLLO-WEB-NET8
```

El proyecto del capítulo está ubicado en:

```
M01_MVC/
  PROYECTO/
    1.1/
      CursoWeb.Mvc/
        CursoWeb.Mvc.csproj
        Program.cs
        Controllers/
          HomeController.cs
        Views/
          _ViewImports.cshtml
          _ViewStart.cshtml
```

```
      Home/
        Index.cshtml
      Shared/
        _Layout.cshtml
    wwwroot/
      css/
        site.css
    appsettings.json
    Properties/
      launchSettings.json
```

Este árbol identifica **todos los archivos necesarios** para el primer recorrido. No necesita base de datos, credenciales ni servicios externos. La práctica no depende del código de ningún otro curso.

Estado inicial para los bloques activables: la cabecera `X-Curso-Paso` está comentada en el controlador y el bloque `marca-paso-1-1-5` está dentro de un comentario Razor. Por tanto, la página se muestra, pero aún no incorpora las dos variaciones.

Mapa verificable de cada paso

Cada fila permite localizar exactamente qué debe consultar o modificar el alumno. **Observar** significa que el código ya está activo y no debe descomentarse; **activar** significa que el proyecto incluye el bloque preparado. Las pruebas E2E ejecutan los estados indicados, pero no reemplazan la observación manual del navegador.

Paso	Archivo y acción del alumno	Resultado y comprobación
1.1.1	Plantilla temporal `.work/PlantillaMvc`. **Acción:** Crear una aplicación desde el SDK; no se entrega como ejercicio activable	**Resultado:** Ejecutar la plantilla en puerto 5095. **Prueba:** Comprobación guiada, fuera del E2E del proyecto
1.1.2	`Program.cs`; `Views/Home/Index.cshtml`. **Acción:** Arrancar el proyecto docente sin editarlo	**Resultado:** GET `/` → 200 HTML. **Prueba:** Estado inicial de `1.1_e2e.sh`
1.1.3	`Program.cs`; `Controllers/HomeController.cs`. **Acción:** Inspeccionar ruta, acción y respuesta sin modificar código	**Resultado:** GET `/` → 200; GET `/css/site.css` → 200 CSS. **Prueba:** Baseline de `1.1_e2e.sh`
1.1.4	`Controllers/HomeController.cs`. **Acción:** Descomentar `Response.Headers["X-Curso-Paso"]`	**Resultado:** Cabecera `X-Curso-Paso: 1.1.4`. **Prueba:** Estado de cabecera en `1.1_e2e.sh`
1.1.5	`Views/Home/Index.cshtml`. **Acción:** Activar el bloque Razor `marca-paso-1-1-5`	**Resultado:** Aparece la sección HTML nueva. **Prueba:** Estado de vista en `1.1_e2e.sh`
1.1.6	`Program.cs`; `wwwroot/css/site.css`. **Acción:** Comparar rutas correctas, archivo estático y 404	**Resultado:** `/Home/Index` → 200, ruta inexistente → 404. **Prueba:** Regresión de rutas en `1.1_e2e.sh`
Reto 1.1	`Controllers/HomeController.cs`; `Views/Home/Index.cshtml`. **Acción:** Activar cabecera y mensaje `reto-1-1`	**Resultado:** Cabecera y HTML presentes de forma independiente. **Prueba:** Escenario del reto de `1.1_e2e.sh`

Regla de trazabilidad. No se considera resuelto un reto solo porque se vea su nombre en el código: se comprueba el resultado esperado con el proyecto activado. Los bloques comentados permanecen como material de ejecución del alumno; no deben sustituirse por capturas ni pseudocódigo.

Desarrollo práctico paso a paso

Paso 1.1.1 — Crear una aplicación ASP.NET Core MVC desde la plantilla

Primero generaremos una aplicación de prueba para identificar la plantilla oficial sin modificar el proyecto docente.

Estando en la **raíz del repositorio**, ejecutar:

```
dotnet new mvc -f net8.0 --no-https -n PlantillaMvc -o .work/PlantillaMvc
```

Qué hace cada opción:

- `dotnet new mvc` utiliza la plantilla de aplicación web MVC instalada con el SDK.
- `-f net8.0` establece el framework de destino.
- `--no-https` simplifica exclusivamente esta primera prueba HTTP local; no se trata de una recomendación de producción.
- `-n PlantillaMvc` nombra el proyecto temporal.
- `-o .work/PlantillaMvc` coloca los archivos en una carpeta descartable excluida de Git.

Para ver que la plantilla es ejecutable:

```
dotnet run --project .work/PlantillaMvc/PlantillaMvc.csproj --no-launch-profile --urls http://
localhost:5095
```

Abrir `http://localhost:5095/`. Debe aparecer la página inicial de la plantilla MVC, con su navegación y su diseño estándar. Detener el proceso con **Ctrl+C**.

Qué acabamos de comprobar: la plantilla genera una aplicación con código servidor, vistas Razor y recursos estáticos. No se ha creado una API REST; se ha creado una web que produce HTML.

El resto de la práctica usará el proyecto **CursoWeb.Mvc** del repositorio, de contenido controlado y vinculado exactamente con este cuaderno. No se debe sustituir ese proyecto por `PlantillaMvc`, porque contienen vistas y nombres distintos.

Paso 1.1.2 — Abrir y ejecutar el proyecto del curso

Desde la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.1/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Alternativa Visual Studio: abrir `CursoWeb.Mvc.csproj`, seleccionar el perfil de ejecución local `CursoWeb.Mvc` (HTTP) y ejecutar. Comprobar el puerto mostrado en la salida. Si se utiliza otro puerto desde Visual Studio, todas las URL siguientes deben adaptarse a ese valor.

Esperar a que la consola indique que el servidor escucha en `http://localhost:5091`. Abrir esa dirección en el navegador.

Resultado esperado visible:

- Un enlace `Inicio` en la cabecera.
- El título `1.1 | Primer recorrido MVC`.

- El texto `Esta página HTML ha sido generada en el servidor mediante una acción MVC.`.
- El mensaje `Respuesta HTTP correcta: aplicación MVC disponible.`.
- Un pie con `Curso ASP.NET Core MVC · .NET 8`.

No debe aparecer todavía `Contenido añadido durante la práctica`. Si aparece, probablemente el bloque del paso 1.1.5 ya esté activado.

Qué ocurre internamente: Kestrel recibe la petición a `/`. La ruta predeterminada selecciona `HomeController.Index()`. La acción devuelve `View()` y MVC renderiza la vista `Views/Home/Index.cshtml`, rodeada por `Views/Shared/_Layout.cshtml`. El navegador recibe el HTML y solicita después la hoja de estilos. Es el primer recorrido extremo a extremo de la aplicación.

Paso 1.1.3 — Inspeccionar petición, ruta y archivos responsables

Con la página abierta, pulsar **F12** (o abrir las herramientas para desarrolladores del navegador), seleccionar **Red/Network** y recargar la página. Si no aparece ninguna fila, mantener abierta la pestaña de red y volver a recargar.

Seleccionar la solicitud de tipo **document** cuyo destino sea `http://localhost:5091/`. Identificar:

1. **Request URL:** `http://localhost:5091/`.
2. **Request Method:** `GET`.
3. **Status Code:** `200`.
4. **Response Headers:** una cabecera `Content-Type` cuyo valor comienza por `text/html`.
5. **Response/Respuesta:** el cuerpo HTML; buscar el texto `Primer recorrido MVC`.

También aparecerá una petición a `/css/site.css`, con tipo de contenido CSS. Esa solicitud es otro recorrido HTTP: el recurso físico está bajo `wwwroot/css/site.css` y lo puede devolver el middleware de archivos estáticos.

Ahora abrir en el editor los cuatro archivos siguientes, **sin modificarlos todavía**:

- `Program.cs`: localizar `AddControllersWithViews()`, `UseStaticFiles()` y `MapControllerRoute(...)`.
- `Controllers/HomeController.cs`: localizar la acción `Index()` y su `return View();`.
- `Views/Home/Index.cshtml`: localizar el `<h1>` que aparece en el navegador.
- `Views/Shared/_Layout.cshtml`: localizar la navegación y `@RenderBody()`.

Seguir mentalmente el recorrido **URL → ruta → controlador → vista → layout → HTML**. Anotar qué archivo determina la URL y qué archivo contiene el texto visible. Es importante diferenciar esos dos conceptos antes de añadir más acciones en 1.4.

Comprobación HTTP alternativa (manteniendo abierta la aplicación, en una segunda terminal):

```
curl -i http://localhost:5091/
```

En Windows PowerShell también puede utilizarse `curl.exe -i http://localhost:5091/`. La salida debe mostrar una línea de estado `200 OK`, cabeceras y HTML. Las cabeceras concretas pueden variar según sistema y servidor; no se exige un orden determinado.

Profundización aplicada: Reconstruir las pruebas desde el navegador.

Este laboratorio amplía la observación de la primera página sin cambiar el proyecto ni añadir tecnologías nuevas. Utilizaremos los mismos fragmentos de código comentados del ejercicio y compararemos lo que revela el navegador con lo que muestra una herramienta de línea de comandos.

Diferenciar el documento principal y los recursos.

Inicia la aplicación del punto 1.1:

```
cd M01_MVC/PROYECTO/1.1/CursoWeb.Mvc
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Con el servidor ejecutándose, abre una pestaña privada del navegador y visita `http://localhost:5091/`. Abre las herramientas de desarrollo con F12, selecciona la pestaña Red o Network y recarga la página.

Localiza la solicitud del documento principal. Debe corresponder a la raíz `/` y devolver el HTML de `Views/Home/Index.cshtml`. No confundas la solicitud del documento con la hoja de estilos `/css/site.css`. Aunque ambas aparezcan asociadas a la misma página visible, sus cuerpos y tipos de contenido son distintos.

Anota para cada solicitud el método, la URL, el código de estado y el tipo de contenido. No hace falta copiar todas las cabeceras: basta con las que permitan explicar qué recurso se pidió y qué respondió el servidor.

Observación	Documento principal	Hoja de estilos
Ruta	`/`	`/css/site.css`
Método	GET	GET
Código esperado	200	200
Contenido esperado	HTML	CSS
¿Interviene HomeController.Index para crear el cuerpo?	Sí	No, lo sirve el middleware estático

La última fila debe justificarse leyendo `Program.cs` y `HomeController.cs`. No es suficiente responder que «ambos son archivos de la aplicación»: el documento principal procede de una acción que devuelve una vista y el CSS se sirve como recurso estático.

Repetir las comprobaciones sin interfaz gráfica.

Mantén la aplicación en ejecución. Abre una segunda terminal y ejecuta por separado:

```
curl -i http://localhost:5091/
curl -i http://localhost:5091/Home/Index
curl -i http://localhost:5091/css/site.css
curl -i http://localhost:5091/ruta-inexistente
```

La opción `-i` incluye las cabeceras HTTP en la salida. La información puede tener distintas cabeceras adicionales según el sistema operativo y la versión del runtime, pero las comprobaciones relevantes son estables para esta práctica: las dos primeras direcciones responden HTML, la tercera CSS y la última 404.

Primera pregunta: ¿por qué hay dos direcciones que llevan al mismo contenido HTML? Consulta la ruta convencional `"{controller=Home}/{action=Index}/{id?}"`. Los valores predeterminados explican que la raíz y `/Home/Index` seleccionen la misma acción.

Segunda pregunta: ¿por qué el recurso CSS responde 200 aunque no exista un método `SiteCss()` en el controlador? Porque el proyecto registra `UseStaticFiles()` y el archivo está ubicado en la raíz web.

Tercera pregunta: ¿la respuesta 404 significa que la aplicación no funciona? No. En este caso significa que la aplicación ha respondido indicando que no se encuentra un endpoint o recurso para la URL solicitada. Las otras direcciones siguen funcionando.

Cuarta pregunta: ¿por qué no es correcto declarar que el navegador ha recibido un objeto C#? Porque el controlador ha seleccionado una vista y Razor ha producido HTML antes de enviar la respuesta.

Documentar una captura de evidencia mínima.

Conserva en tus apuntes una pequeña tabla con cuatro filas, una por solicitud, y apunta el código de estado y el tipo de contenido observados. No es necesario incorporar capturas de todos los paneles del navegador. El objetivo es que otra persona pueda reproducir el recorrido utilizando exactamente las mismas URLs.

Si alguna solicitud no coincide con lo previsto, identifica primero si el problema afecta al servidor, a la ruta o al cuerpo de la respuesta. Evita modificar varios archivos al mismo tiempo.

Paso 1.1.4 — Activar una cabecera HTTP desde la acción

Dejar de ejecutar la aplicación mediante **Ctrl+C** antes de modificar y volver a construir. Abrir `Controllers/HomeController.cs` y localizar:

```
// PASO 1.1.4 — Descomentar solo la siguiente línea para observar
// una cabecera de respuesta HTTP generada desde el controlador.
// Response.Headers["X-Curso-Paso"] = "1.1.4";
```

Quitar las dos barras `//` **solo** delante de la asignación; las líneas explicativas permanecen comentadas. El fragmento activo quedará así:

```
Response.Headers["X-Curso-Paso"] = "1.1.4";
return View();
```

Guardar y ejecutar de nuevo:

```
dotnet run --no-launch-profile --urls http://localhost:5091
```

Recargar la página y observar las cabeceras de **respuesta** en F12 → Red. Debe aparecer:

```
X-Curso-Paso: 1.1.4
```

También se puede comprobar con `curl -i http://localhost:5091/`.

Qué hemos aprendido: la cabecera se añade desde el controlador a la respuesta HTTP, pero no altera el título ni el texto HTML. Si solo miramos la pantalla, podemos pensar incorrectamente que el cambio no se ha aplicado.

Comprobación negativa: la cabecera no tiene por qué aparecer en la petición CSS, porque ese archivo estático no lo sirve la acción `Index()`. La respuesta de la página y la respuesta de la hoja de estilos proceden de solicitudes diferentes.

Profundización aplicada: Comparar cambios de controlador y cambios de Razor.

En esta actividad usarás dos bloques ya existentes y comentados en el proyecto. No añadas archivos nuevos. El código aparece aquí para poder comparar exactamente qué se activa en cada estado.

Estado inicial sin fragmentos activados.

Abre `Controllers/HomeController.cs` y localiza:

```
// Response.Headers["X-Curso-Paso"] = "1.1.4";
```

Después abre `Views/Home/Index.cshtml` y localiza:

```
@* PASO 1.1.5 (inicio)
<section id="marca-paso-1-1-5">
    <h2>Contenido añadido durante la práctica</h2>
    <p>El navegador ha recibido una nueva versión de la vista Razor.</p>
</section>
PASO 1.1.5 (fin) *@
```

Ambos fragmentos están preparados. El primero está comentado con `//` porque es C#. El segundo utiliza delimitadores Razor `@* ... *@` porque es una plantilla `.cshtml`. Este detalle no es puramente visual: quitar los delimitadores incorrectos puede generar código que no compila o que Razor no representa como esperamos.

Antes de activarlos, comprueba que la cabecera `X-Curso-Paso` no aparece en la respuesta del documento y que tampoco aparece el elemento `id="marca-paso-1-1-5"` en el HTML. Estos dos resultados negativos permiten reconocer luego qué cambió.

Activar únicamente el controlador.

Retira las dos barras de comentario de la asignación `Response.Headers`, guarda el archivo, detén y vuelve a ejecutar la aplicación. No actives todavía el bloque Razor.

En otra terminal:

```
curl -i http://localhost:5091/
```

Busca la cabecera `X-Curso-Paso: 1.1.4`. A continuación abre el navegador y observa el contenido visible. El encabezado y los párrafos pueden parecer exactamente iguales que en el estado anterior; eso no invalida el resultado porque la modificación está en las cabeceras de la respuesta.

Anota el dato más importante: **la instrucción del controlador se ha ejecutado aunque no se haya añadido ningún elemento HTML**. El alumnado debe poder señalar la línea concreta que explica la cabecera.

Para comprobar el alcance de la modificación, solicita:

```
curl -i http://localhost:5091/css/site.css
```

No debes encontrar allí la cabecera personalizada de Home, porque la petición CSS la atiende el middleware estático. Si la encuentras, revisa que no hayas añadido otra instrucción global fuera del ejercicio.

Activar únicamente la vista.

Vuelve a comentar la asignación `X-Curso-Paso` y activa ahora el bloque `PASO 1.1.5`, retirando las dos líneas delimitadoras del comentario Razor. Compila y vuelve a ejecutar.

La cabecera de diagnóstico no debería aparecer ya. En cambio, el HTML debe incluir una sección con el identificador `marca-paso-1-1-5` y el mensaje sobre la nueva versión de la vista.

En F12 → Elementos o Inspector busca el `section`. Compara esa estructura con el código del archivo `Index.cshtml`. Después examina la respuesta del documento en la pestaña Red y comprueba que el

fragmento forma parte del HTML entregado, no de una modificación realizada por código JavaScript del cliente.

El cambio ha afectado a la representación visual de la página. No ha sido necesario tocar `Program.cs` ni la ruta convencional.

Activar ambos cambios y explicar la independencia.

Activa de nuevo la cabecera, conservando también activo el bloque de la vista. Después de compilar y ejecutar, comprueba ambos resultados en la misma respuesta: la cabecera `X-Curso-Paso` en la zona de metadatos y el `section` en el cuerpo HTML.

Si solo aparece uno, no deberías cambiar el otro fragmento a ciegas. Revisa el archivo responsable del elemento ausente y confirma que estás ejecutando la versión correcta del proyecto.

Esta práctica controlada puede representarse como una matriz:

Estado	Cabecera HTTP	Elemento HTML
Inicial	Ausente	Ausente
Controlador	Presente	Ausente
Vista	Ausente	Presente
Ambos	Presente	Presente

La batería `1.1_e2e.sh` reproduce estas combinaciones mediante copias temporales, lo que permite comprobarlas sin modificar el proyecto fuente del alumno.

Paso 1.1.5 — Activar un bloque HTML de la vista Razor

Detener de nuevo la aplicación. Abrir `Views/Home/Index.cshtml`. Al final aparece este fragmento:

```
@* PASO 1.1.5 (inicio)
<section id="marca-paso-1-1-5">
    <h2>Contenido añadido durante la práctica</h2>
    <p>El navegador ha recibido una nueva versión de la vista Razor.</p>
</section>
PASO 1.1.5 (fin) *@
```

En Razor, `@*` abre un comentario de servidor y `*@` lo cierra. **Mientras los delimitadores sigan presentes, el bloque HTML no llegará al navegador**.

Eliminar completamente la línea `@* PASO 1.1.5 (inicio)` y la línea `PASO 1.1.5 (fin) *@`, pero **conservar el bloque `<section>` interior**. Debe quedar:

```
<section id="marca-paso-1-1-5">
    <h2>Contenido añadido durante la práctica</h2>
    <p>El navegador ha recibido una nueva versión de la vista Razor.</p>
</section>
```

Guardar, reiniciar `dotnet run` y recargar. En la página, debajo del mensaje inicial, debe verse **Contenido añadido durante la práctica**. En F12 → Red → Response debe aparecer también el elemento `<section id="marca-paso-1-1-5">`.

Qué hemos aprendido: modificar una vista modifica el cuerpo HTML de la respuesta. La URL `/` y la acción `Index()` siguen siendo las mismas; ahora el navegador dispone de más elementos para representar.

Los pasos 1.1.4 y 1.1.5 son **compatibles**. Se pueden mantener activados simultáneamente: el primero añade una cabecera, mientras que el segundo añade contenido.

Paso 1.1.6 — Comprobar una ruta válida alternativa y un 404

Con la aplicación ejecutándose, abrir estas direcciones:

```
http://localhost:5091/
http://localhost:5091/Home/Index
http://localhost:5091/ruta-inexistente
http://localhost:5091/css/site.css
```

La primera y la segunda URL deben mostrar **la misma página**. La segunda expresa explícitamente controlador y acción; la primera utiliza los valores por defecto de la ruta convencional.

La tercera debe responder con **404 Not Found**, porque no tenemos una acción correspondiente a `ruta-inexistente`. Es normal que su apariencia sea distinta de la de la página válida: el servidor ha respondido, pero no dispone de una página para ese destino.

La cuarta debe mostrar el texto CSS, no una vista MVC. Esto confirma que una aplicación ASP.NET Core puede servir distintos tipos de recursos HTTP.

Con una segunda terminal, comprobar el 404 sin depender de cómo lo representa el navegador:

```
curl -i http://localhost:5091/ruta-inexistente
```

Si los códigos de respuesta no coinciden con lo esperado, revisar el puerto, la URL exacta y que se esté ejecutando el proyecto `CursoWeb.Mvc` en lugar de `PlantillaMvc`.

Reto: aplicación de los pasos

Consolidación y reto 1.1 — distinguir metadatos HTTP y HTML.

Observación y razonamiento.

Con la aplicación funcionando, verifica que `/` y `/Home/Index` corresponden a la misma acción; compara el documento HTML con `/css/site.css`; explica por qué una URL inexistente recibe 404 sin que falle el servidor. Identifica qué cambia cuando activas el paso 1.1.4 (cabecera) y el 1.1.5 (cuerpo HTML).

Reto práctico — agregar una segunda cabecera y un texto de diagnóstico.

Objetivo: modificar simultáneamente los metadatos de respuesta y el contenido de una vista, demostrando que pertenecen a responsabilidades diferentes. El código del reto **ya está en el proyecto**, pero permanece comentado para que puedas activarlo. Primero resuelve el reto y después compara con el código proporcionado.

Preparación: utiliza `M01_MVC/PROYECTO/1.1/CursoWeb.Mvc` y deja activados los pasos 1.1.4 y 1.1.5. Detén la aplicación antes de editar.

Acción 1 — controlador. En `Controllers/HomeController.cs`, localiza las líneas marcadas `RETO 1.1`. Descomenta exclusivamente la asignación:

```
// RETO 1.1 — Cambiar una cabecera de respuesta sin tocar la vista.
Response.Headers["X-Curso-Reto"] = "HTTP-y-HTML";
```

Esta instrucción añade la cabecera `X-Curso-Reto` a la respuesta HTTP del documento principal, sin modificar el HTML y sin afectar a la respuesta del CSS. Almacena la cadena «HTTP-y-HTML» como valor de la cabecera, no como texto de pantalla.

Acción 2 — vista. En `Views/Home/Index.cshtml`, busca el bloque `RETO 1.1 (inicio)`. Elimina **solo** las dos líneas que delimitan el comentario Razor `@* ... *@`, dejando este HTML activo:

```
<p id="reto-1-1">El servidor genera el HTML y envía las cabeceras HTTP por separado.</p>
```

El atributo `id` facilita localizar el nuevo elemento en las herramientas de desarrollo y en las pruebas automatizadas. El texto se enviará dentro del cuerpo HTML y aparecerá bajo el contenido existente. La vista no está añadiendo por sí misma ninguna cabecera HTTP.

Acción 3 — ejecutar. Desde la carpeta del proyecto:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Accede a `http://localhost:5091/`. Debes ver el texto «El servidor genera el HTML y envía las cabeceras HTTP por separado». En F12 → Red → solicitud del documento → cabeceras de respuesta, comprueba `X-Curso-Reto: HTTP-y-HTML`. También puedes ejecutar `curl -i http://localhost:5091/` desde otra terminal.

Acción 4 — contrastar. Visita `/css/site.css` y verifica que no aparece la cabecera personalizada; responde una solicitud atendida por archivos estáticos, no por `HomeController.Index()`.

Código de solución y activación: las instrucciones anteriores contienen la solución completa para comparar con tu intento. No hay que crear nuevos archivos ni copiar código externo. El reto se activa **sobre el proyecto que ya contiene todos los bloques identificados**.

Prueba automatizada del reto: desde la raíz del repositorio, `bash M01_MVC/TESTS/1.1_e2e.sh` incluye el estado `reto`. Ejecuta en una copia temporal la aplicación con los ejercicios anteriores y el reto activados; verifica que compila, que devuelve HTTP 200, que existe `X-Curso-Reto` y que el párrafo aparece únicamente al activar la variante.

Resultado esperado: distinguir, señalando el archivo y la línea concreta, un cambio de cabeceras (controlador) de un cambio de representación (vista). El reto tiene una solución guiada y comprobable; no implica nuevos conceptos fuera de 1.1.

Resolver el reto con dos comprobaciones independientes.

El reto 1.1 ya está desarrollado en el apartado G. Aquí ampliamos la comprobación para que su resultado no dependa únicamente de observar un mensaje visible.

Código de la cabecera del reto.

En `HomeController.cs` permanece comentada esta instrucción:

```
// Response.Headers["X-Curso-Reto"] = "HTTP-y-HTML";
```

Retira el prefijo `//` para activarla. Al ejecutarse la acción Home, la respuesta del documento contendrá la cabecera `X-Curso-Reto`.

Código de la vista del reto.

En `Views/Home/Index.cshtml` encontramos:

```
@* RETO 1.1 (inicio)
<p id="reto-1-1">El servidor genera el HTML y envía las cabeceras HTTP por separado.</p>
RETO 1.1 (fin) *@
```

Retira las dos líneas de comentario Razor y conserva el párrafo. El atributo `id` ayuda a encontrar el elemento en el HTML y a comprobar su presencia mediante el script E2E.

Tres solicitudes para comprobar el comportamiento.

Con ambos fragmentos activos, ejecuta:

```
curl -i http://localhost:5091/
curl -i http://localhost:5091/Home/Index
curl -i http://localhost:5091/css/site.css
```

Las dos primeras solicitudes llegan a la misma acción y deben incorporar el resultado del reto. La tercera accede al recurso CSS y no debe incorporar la cabecera añadida dentro de Home.

En las respuestas HTML, busca el párrafo `reto-1-1`. No debería encontrarse en el contenido de la hoja CSS, porque no estamos sirviendo una vista Razor mediante esa solicitud.

Explicación de cierre.

Una solución completa del reto debe identificar **dos cambios distintos**. El primero corresponde al controlador y modifica una cabecera. El segundo corresponde a la vista y modifica el HTML. Ambos forman parte de la misma respuesta del documento, pero no son el mismo tipo de información.

El criterio de finalización no es simplemente «todo funciona»: debes poder explicar por qué esos cambios son independientes y por qué no afectan de la misma manera a los recursos estáticos.

Comprobaciones funcionales y casos de diagnóstico

Ejecución completa y resultados esperados.

Al terminar los cambios, mantener el proceso MVC en ejecución con `dotnet run --no-launch-profile --urls http://localhost:5091`. Al visitar `/` deben coexistir: un código HTTP **200**, la cabecera `X-Curso-Paso: 1.1.4` y el contenido HTML añadido en el paso 1.1.5. La hoja de estilos debe cargarse; la URL desconocida debe responder 404.

Este es el comportamiento observable que prueba que los dos fragmentos están activados y que el recorrido principal funciona.

Comprobación funcional del alumno.

Caso que se debe probar	Operación	Resultado correcto
Plantilla oficial	Abrir el sitio temporal en 5095 tras `dotnet new mvc`	Página de bienvenida MVC
Aplicación del curso	GET `/` en 5091	200 y encabezado `Primer recorrido MVC`
Cabecera del controlador	GET `/` tras activar 1.1.4	`X-Curso-Paso: 1.1.4`

Caso que se debe probar	Operación	Resultado correcto
Contenido de la vista	GET `/` tras activar 1.1.5	Visible `Contenido añadido durante la práctica`
Ruta alternativa	GET `/Home/Index`	200 y mismo contenido
Ruta inexistente	GET `/ruta-inexistente`	404
Recurso estático	GET `/css/site.css`	200 y contenido CSS

La prueba automatizada del proyecto comprueba los recorridos HTTP de la aplicación del curso, incluidos los estados base y activado. Se puede ejecutar desde la raíz del repositorio con `bash M01_MVC/TESTS/1.1_e2e.sh` si se dispone de Bash, curl, Python 3 y SDK 8. La creación de la plantilla temporal es una operación didáctica guiada y no modifica la aplicación distribuida.

Criterio de finalización: el alumno debe poder señalar, sobre una petición real, dónde están el método GET, el código 200/404, el encabezado de respuesta, la acción MVC y el HTML de la vista.

Resolución de incidencias

Problemas frecuentes y soluciones.

La terminal indica que `dotnet` no existe. El SDK no está instalado o no está disponible en el PATH. Instalar o reparar SDK .NET 8 y comprobar `dotnet --list-sdks` antes de continuar.

`dotnet new mvc -f net8.0` no reconoce el framework. Comprobar con `dotnet --info` que el SDK 8 está instalado y que no existe un `global.json` que obligue a otro SDK incompatible. No cambiar a net9.0 o net10.0 para continuar el curso.

El puerto 5091 está ocupado. Cerrar otra instancia de la misma aplicación o identificar qué proceso usa el puerto. Es posible seleccionar temporalmente otro puerto mediante `--urls`, pero en ese caso habrá que actualizar todas las URL de la práctica.

Se abre la plantilla en lugar de la página del curso. Se está ejecutando `PlantillaMvc` en el puerto 5095 o se abrió otro proyecto. Detener esa aplicación y arrancar desde `M01_MVC/PROYECTO/1.1/CursoWeb.Mvc` en 5091.

El navegador muestra una pantalla de error de conexión. A diferencia de un HTTP 404, eso suele significar que no existe un servidor escuchando en ese puerto o que se cerró el proceso. Comprobar la consola de `dotnet run`.

No aparece la cabecera personalizada. Comprobar que se quitó el comentario de la línea correcta, que el archivo se guardó, que el proceso se reconstruyó y que se está inspeccionando la respuesta de `/`, no la de `site.css`.

No aparece el contenido adicional en la vista. Revisar que se quitaron *ambos* delimitadores de comentario Razor, que se mantuvo intacto el elemento `<section>` y que se recargó el documento. Para descartar una copia anterior usar una actualización completa en el navegador.

Se recibe un 404 en `/Home/Index`. Verificar que existen `HomeController` e `Index()`, el patrón de `MapControllerRoute` y que se ejecuta la aplicación del curso. En 1.4 veremos cómo diagnosticar errores de rutas con mayor detalle.

En Windows, `curl` parece devolver un objeto en vez de texto HTTP. Dependiendo de la versión de PowerShell, `curl` puede ser un alias de otro comando. Ejecutar `curl.exe -i http://localhost:5091/` o usar la pestaña Red del navegador.

Interpretar errores previsibles.

Error: el servidor no se inicia.

Si aparece un error relacionado con el uso del puerto o no puedes abrir `localhost:5091`, verifica que no esté funcionando otra versión del proyecto sobre la misma dirección. Detén procesos anteriores y ejecuta de nuevo desde la carpeta correcta.

No cambies el contenido Razor para resolver un conflicto de puerto. La causa se encuentra antes de que la solicitud llegue a MVC.

Error: la ruta alternativa funciona pero la raíz no.

Si `/Home/Index` funciona y `/` no, compara la configuración de `MapControllerRoute` con el código proporcionado. La convención del ejercicio utiliza `Home` e `Index` como valores predeterminados. Si alguno se cambió, puede variar el resultado de la raíz.

Antes de introducir otra ruta, restaura el código original y comprueba la diferencia entre el estado que fallaba y la configuración inicial.

Error: la página funciona sin estilos.

Comprueba `/css/site.css` directamente. Si devuelve 404, revisa la carpeta `wwwroot/css` y la presencia de `UseStaticFiles`. Si responde correctamente pero el navegador no aplica la hoja, inspecciona el atributo `href` del enlace CSS en `_Layout.cshtml` y verifica que el recurso no se encuentre bloqueado o sustituido por otro estilo.

Es importante registrar **qué comprobación concreta falló**. La frase «no tiene estilos» no permite elegir una única solución sin observar la solicitud.

Error: el fragmento Razor aparece como texto raro o da un error de vista.

Revisa que se hayan eliminado únicamente las dos líneas delimitadoras del comentario Razor. El cuerpo de `<section>` y sus etiquetas deben permanecer tal como están preparados en el proyecto.

No intentes resolverlo suprimiendo todas las marcas `@` del archivo: algunas pertenecen a Razor y son necesarias para el contenido de la vista.

Error: el cambio está guardado pero no aparece.

Comprueba la carpeta del proyecto. El curso conserva varias versiones con nombres similares porque cada punto debe poder ejecutarse independientemente. Si editas el archivo en `PROYECTO/1.1` pero ejecutas la aplicación de `PROYECTO/1.2`, la modificación no tendrá efecto en la versión que responde a las solicitudes.

El camino correcto de diagnóstico es detener el proceso, comprobar el directorio de trabajo, compilar el proyecto esperado y volver a ejecutar.

Resultado y continuidad

Continuidad del proyecto.

Conservar el proyecto CursoWeb.Mvc y los archivos modificados. En los siguientes capítulos añadiremos modelos de productos, nuevas acciones y vistas sin repetir toda la creación inicial. No introducimos EF Core todavía: aparecerá en el día 2. La aplicación temporal `.work/PlantillaMvc` se puede eliminar una vez comprendida la plantilla.

Síntesis de 1.1 para estudiar sin el profesor.

Antes de continuar con 1.2, verifica que puedes ejecutar el proyecto desde una terminal limpia y repetir el recorrido sin mirar una captura de pantalla de la clase. Debes saber qué comando inicia el servidor, qué URL corresponde a la portada y dónde está el controlador que devuelve la vista.

Después, localiza un archivo HTML de vista y una hoja CSS, y describe por qué el navegador los obtiene mediante mecanismos distintos. No es suficiente señalar carpetas: identifica la acción MVC y el middleware estático que intervienen.

Finalmente, activa de nuevo los bloques del ejercicio y del reto, y comprueba que puedes atribuir cada diferencia al archivo correcto. Si eres capaz de explicar qué cambia al modificar el controlador, qué cambia al modificar Razor y qué sigue funcionando sin cambios, has alcanzado el objetivo de esta primera práctica.

Relación con la siguiente práctica: en 1.2 ya no nos limitaremos a representar un HTML fijo. Introduciremos una clase `Producto` y una colección que la acción entrega a la vista. El método de observación seguirá siendo el mismo: un cambio controlado, un resultado visible y una comprobación que relaciona ese resultado con el código.

P1.2. El flujo Modelo–Vista–Controlador

Objetivo y preparación

Objetivo. Comprender la función de los tres elementos del patrón MVC, ejecutando el recorrido completo de una petición hacia un controlador de productos, entregando una colección de modelos a la vista y activando progresivamente su presentación y navegación.

Teoría de referencia. Capítulo 1.2 de `M01_TEORIA.md`: responsabilidades, comunicación entre piezas, `View(productos)`, vistas tipadas y separación de datos frente a marcado.

Conocimientos previos. Se presupone completado el punto 1.1 y conocimiento de listas y clases sencillas de C#. No se necesita EF Core ni ninguna base de datos.

Resultado concreto. Una página `/Productos` que presenta la cantidad de productos y tres registros con su nombre y precio, enlazada desde la cabecera del sitio. Los datos se mantienen exclusivamente en memoria.

Duración de referencia. El plan docente reserva 25 minutos de explicación y unos 35 minutos de trabajo práctico. Se ofrece material de lectura y comprobación detallado que también puede utilizarse fuera del aula; dentro de la sesión, el alumnado realizará los cambios indicados y verificará los resultados más relevantes.

El material se construye como **segunda evolución del mismo proyecto MVC**; para que pueda abrirse y probarse sin depender de copias manuales, la versión del capítulo tiene una carpeta propia:

```
M01_MVC/
  PROYECTO/
    1.1/
      CursoWeb.Mvc/           # Estado conservado del capítulo anterior
    1.2/
      CursoWeb.Mvc/           # Continuación del mismo proyecto, con material del punto 1.2
        CursoWeb.Mvc.csproj
        Program.cs
        Models/
          Producto.cs
        Controllers/
          HomeController.cs
          ProductosController.cs
        Views/
          Home/
            Index.cshtml
          Productos/
            Index.cshtml
          Shared/
            _Layout.cshtml
          _ViewStart.cshtml
          _ViewImports.cshtml
        wwwroot/
          css/
            site.css
        ...
```

No editar el proyecto 1.1 para realizar este capítulo. El proyecto 1.2 contiene su funcionalidad anterior y los nuevos archivos necesarios. Cuando terminemos M01, las sucesivas versiones formarán una secuencia didáctica verificable.

Desde la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.2/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/` y comprobar que sigue apareciendo la página de bienvenida del 1.1. Es la primera comprobación de **no regresión**: introducir un controlador nuevo no debe estropear el existente.

Abrir después `http://localhost:5091/Productos`. Debe aparecer:

- Título: **1.2 | Modelo, Vista y Controlador**.
- Un texto que explica que la vista presenta datos.
- Mensaje: **El controlador todavía no ha entregado una lista a la vista**.

En este estado, la dirección es válida y responde 200, pero aún no se ha activado la entrega de una lista al resultado de vista.

Antes de seguir, detener la aplicación con **Ctrl+C**; reiniciaremos el proceso entre estados. El alumno puede dejar la terminal abierta en la carpeta del proyecto.

Mapa verificable de cada paso

Cada fila permite localizar exactamente qué debe consultar o modificar el alumno. **Observar** significa que el código ya está activo y no debe descomentarse; **activar** significa que el proyecto incluye el bloque preparado. Las pruebas E2E ejecutan los estados indicados, pero no reemplazan la observación manual del navegador.

Paso	Archivo y acción del alumno	Resultado y comprobación
1.2.1	`Program.cs`; `Controllers/ProductosController.cs`. **Acción:** Examinar la ruta del catálogo ya definida	**Resultado:** `/Productos` → 200; modelo todavía nulo. **Prueba:** Estado inicial de `1.2_e2e.sh`
1.2.2	`Models/Producto.cs`. **Acción:** Leer las propiedades del modelo existente, sin descomentar nada	**Resultado:** Identificar `Id`, `Nombre` y `Precio`. **Prueba:** Compilación de `1.2_e2e.sh`
1.2.3	`Controllers/ProductosController.cs`. **Acción:** Activar el bloque de lista y `View(productos)`; comentar el retorno inicial	**Resultado:** `/Productos` muestra un contador de 3. **Prueba:** Estado de datos de `1.2_e2e.sh`
1.2.4	`Views/Productos/Index.cshtml`. **Acción:** Activar el bucle Razor de `listado-productos`	**Resultado:** Tres elementos con nombre, precio e identificador. **Prueba:** Estado del listado de `1.2_e2e.sh`
1.2.5	`Views/Shared/_Layout.cshtml`. **Acción:** Activar enlace a `/Productos`	**Resultado:** Menú Inicio/Productos navegable. **Prueba:** Estado de navegación de `1.2_e2e.sh`
1.2.6	`Program.cs`; vistas y controlador anteriores. **Acción:** Repasar el flujo completo sin crear una segunda implementación	**Resultado:** Se mantienen portada y catálogo 200. **Prueba:** Regresión final de `1.2_e2e.sh`
Reto 1.2	`Controllers/ProductosController.cs`. **Acción:** Sustituir Cuaderno por Bloc en el bloque indicado, no duplicarlo	**Resultado:** Tres productos; Bloc sustituye a Cuaderno. **Prueba:** Escenario de reto de `1.2_e2e.sh`

Regla de trazabilidad. No se considera resuelto un reto solo porque se vea su nombre en el código: se comprueba el resultado esperado con el proyecto activado. Los bloques comentados permanecen como material de ejecución del alumno; no deben sustituirse por capturas ni pseudocódigo.

Desarrollo práctico paso a paso

Paso 1.2.1. Seguir la petición hasta el controlador

Abrir `Program.cs` y localizar la llamada:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Cuando la URL es `/Productos`, el sistema de rutas interpreta que el segmento `Productos` representa el controlador, mientras que `Index` es la acción predeterminada. El método situado en `Controllers/ProductosController.cs` es:

```
[HttpGet]
public IActionResult Index()
{
    // Dentro encontraremos los bloques de código del ejercicio.
    return View();
}
```

Este fragmento muestra solamente las instrucciones activas que explican el comportamiento inicial. La versión real contiene, además, el bloque comentado que habilitaremos en un paso posterior.

Comprueba la relación con la vista abriendo `Views/Productos/Index.cshtml`. Es la ruta convencional que utiliza `return View();` desde la acción `Index` del controlador `ProductosController`. La vista muestra un mensaje porque todavía no recibe ningún dato.

Anota estas tres correspondencias:

Elemento	Ruta o tipo del proyecto	Responsabilidad
URL	`/Productos`	Petición del navegador
Controlador	`Controllers/ProductosController.cs`	Atiende la acción `Index`
Vista	`Views/Productos/Index.cshtml`	Genera la presentación HTML

Comprobación: el navegador debe mostrar una página de `Productos` aunque la lista esté vacía por ausencia de modelo. Un 200 con mensaje no es un 404.

Profundización aplicada: Comprender las rutas convencionales.

La aplicación utiliza la misma ruta convencional estudiada en 1.1. Ahora existe un controlador adicional llamado `ProductosController` con una acción `Index()`.

Arranca la aplicación con los pasos 1.2.3 y 1.2.4 activos y realiza estas solicitudes:

```
curl -i http://localhost:5091/Productos
curl -i http://localhost:5091/Productos/Index
curl -i http://localhost:5091/
curl -i http://localhost:5091/ruta-inexistente
```

Las dos primeras deben mostrar la misma página de productos. La tercera sigue mostrando la portada; la cuarta responde 404. Explica por qué añadir un nuevo controlador no ha eliminado ni sustituido automáticamente al anterior.

En `Program.cs` identifica los valores predeterminados Home e Index. Cuando la URL comienza por `/Productos`, la primera posición selecciona Productos en lugar de Home. Cuando omitimos la acción, se utiliza Index.

Comprobación de regresión: pulsa Inicio si el enlace se encuentra disponible o escribe directamente `/`. La página anterior debe continuar funcionando después de activar el nuevo modelo y la vista. Si no sucede, es importante revisar qué archivos se han modificado además de los previstos.

Paso 1.2.2. Reconocer la clase que representa un producto

Abrir `Models/Producto.cs` y examinar la clase que se ha suministrado:

```
namespace CursoWeb.Mvc.Models;

public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
}
```

Qué debes identificar: `Id` distingue un producto de otro; `Nombre` recoge el texto que verá el usuario; `Precio` almacena una cantidad decimal. La clase no importa `Microsoft.AspNetCore.Mvc`, no usa `HttpContext` y no conoce archivos `.cshtml`.

No es necesario descomentar nada en este archivo: es un tipo C# válido y **ya está activo**. El objetivo del paso es reconocer el contrato de datos con el que trabajarán el controlador y la vista. La primera página del proyecto puede continuar funcionando aunque esta clase exista y aún no se utilice.

Pregunta de comprobación: ¿qué propiedad tendrías que consultar para mostrar el nombre de un producto? ¿Sería necesario acceder al navegador desde la clase? Debes poder responder `Nombre` y «no», respectivamente.

Profundización aplicada: Reconstruir el flujo de datos.

El recorrido básico del punto 1.2 ya funciona cuando se activan los pasos 1.2.3 y 1.2.4. Este laboratorio complementario permite comprenderlo con mayor precisión, sin añadir tipos ni proyectos que no estén incluidos en el curso.

Localizar el contrato antes de ejecutar la aplicación.

Abre `Models/Producto.cs` y copia en tus apuntes la definición de sus tres propiedades:

```
public int Id { get; set; }
public string Nombre { get; set; } = string.Empty;
public decimal Precio { get; set; }
```

Para cada propiedad anota su tipo y qué información representa. Después abre `Controllers/ProductosController.cs` y localiza la creación de la colección:

```
var productos = new List<Producto>
{
    new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
    new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
    new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
};
```

El controlador crea tres instancias. No se conecta a SQLite ni utiliza EF Core. Ese hecho debe quedar claro desde el principio, porque el ejercicio está dedicado a seguir el flujo entre componentes MVC y no a enseñar persistencia.

En `Views/Productos/Index.cshtml` identifica la declaración:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

Pregunta de comprobación: ¿por qué la vista puede recibir una colección `List<Producto>` si ha declarado `IReadOnlyList<Producto>`? Porque una lista de C# implementa esa interfaz. La vista declara las operaciones que necesita para consultar los datos, sin exigir que el controlador utilice exactamente el mismo tipo concreto.

Analizar el retorno de la acción.

Con el paso 1.2.3 activado, la instrucción principal del controlador es:

```
return View(productos);
```

A partir de esta línea, responde las siguientes preguntas sin ejecutar todavía:

- ¿Se devuelve una cadena de HTML directamente? No: se crea un resultado de vista al que se entrega el modelo.
- ¿Se ha seleccionado una vista llamada `Productos` mediante un nombre explícito? No: al usar `View(productos)`, MVC aplica sus convenciones de búsqueda relacionadas con la acción `Index` del controlador `ProductosController`.
- ¿Se envía una única instancia `Producto`? No: la variable contiene una lista de tres objetos.
- ¿La vista necesita volver a crear esos tres objetos? No: los recibe como `Model`.

A continuación ejecuta la aplicación y abre `/Productos`. Comprueba que el contador coincide con el número de inicializadores `new()` activos en la lista.

Diferenciar una colección nula de una colección con elementos.

La vista contiene una condición:

```
@if (Model is null)
{
    <p id="sin-modelo">El controlador todavía no ha entregado una lista a la vista.</p>
}
else
{
    <p id="numero-productos">El controlador ha enviado <strong>@Model.Count</strong> productos.</p>
}
```

El ejemplo es una representación abreviada: en el archivo real, el `else` contiene también el fragmento de listado que se activa después. En el estado inicial, cuando la acción utiliza `View()`, aparece el mensaje `sin-modelo`. Cuando activa `View(productos)`, el modelo deja de ser nulo y la vista muestra el contador.

Compara las dos situaciones anotando **qué línea del controlador provoca el cambio**. No debes atribuirlo a una modificación en la ruta; ambas peticiones usan `/Productos`.

La condición permite además razonar sobre un caso que no activaremos: una lista vacía. Si el controlador entregara una lista sin elementos, `Model` existiría y su `Count` sería cero. El `else` seguiría

ejecutándose y el bucle de representación no produciría filas. Este análisis prepara al alumno para las situaciones de catálogo vacío en aplicaciones reales.

Paso 1.2.3. Activar el código que entrega modelos a la vista

Abrir `Controllers/ProductosController.cs`. Dentro del método `Index` localizar el siguiente bloque, que se proporciona comentado:

```
/* PASO 1.2.3 INICIO
var productos = new List<Producto>
{
    new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
    new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
    new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
};

return View(productos);
PASO 1.2.3 FIN */

return View(); // PASO 1.2.3: comentar al activar el envío de datos.
```

En el archivo real, cada línea está indentada dentro del método. No cambies el contenido ni su posición.

Acción 1: eliminar completamente **las dos líneas** que empiezan por `/* PASO 1.2.3 INICIO` y `PASO 1.2.3 FIN */`. Conservar las líneas que crean la lista y devuelven `View(productos)`.

Acción 2: comentar la **última** instrucción `return View();` que quedaba activa en la acción. No basta con quitar los delimitadores: si dejamos ambas instrucciones de retorno activas, la segunda será inalcanzable y habremos dejado código confuso.

El resultado funcional debe tener esta forma:

```
[HttpGet]
public IActionResult Index()
{
    var productos = new List<Producto>
    {
        new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
        new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
        new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
    };

    return View(productos);

    // return View(); // PASO 1.2.3: comentar al activar el envío de datos.
}
```

Explicación:

1. `new List<Producto>` crea una colección en memoria. Todavía no hay base de datos.
2. Cada inicializador `new()` crea un objeto de tipo `Producto`; C# conoce su tipo por la declaración de la colección.
3. Los valores `4.50m`, `1.20m` y `3.75m` son cantidades decimales.
4. `View(productos)` selecciona la vista y le proporciona esa lista como modelo. El controlador no genera manualmente un documento HTML.

Guardar y comprobar desde la carpeta del proyecto:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Visitar `http://localhost:5091/Productos`. El mensaje sin modelo debe haber desaparecido y debe mostrarse: **El controlador ha enviado 3 productos**. Todavía **no** aparece la lista de nombres. Esto es correcto: el controlador ya ha entregado información, pero el bloque que la representa detalladamente sigue comentado.

Detener el servidor antes de continuar.

Paso 1.2.4. Activar la vista que representa los productos

Abrir `Views/Productos/Index.cshtml`. En la primera línea observarás:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

La directiva `@model` especifica que la vista espera una lista de productos de solo lectura o, en el estado inicial, un modelo `null`. Esto no impide que el controlador construya una `List<Producto>` normal: dicha lista implementa `IReadOnlyList<Producto>`.

Localizar el bloque HTML y Razor del ejercicio:

```
@* PASO 1.2.4 (inicio)
<ul id="listado-productos">
@foreach (var producto in Model)
{
    <li data-id="@producto.Id">@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
}
</ul>
PASO 1.2.4 (fin) *@
```

Acción: eliminar únicamente las dos líneas delimitadoras `@* PASO 1.2.4 (inicio)` y `PASO 1.2.4 (fin) *@`. Mantener intacto el bloque `<ul>` y su `@foreach`.

En la versión real, el bloque está dentro del `else` que se ejecuta cuando `Model` no es `null`. No saques el `@foreach` de ese `else`: mantener la estructura permite al ejemplo distinguir correctamente los dos estados.

Después de guardar y ejecutar:

```
dotnet run --no-launch-profile --urls http://localhost:5091
```

Acceder de nuevo a `/Productos`. Ahora deben aparecer el contador y **los tres nombres**: Cuaderno, Lápiz y Carpeta, cada uno con su precio. Puede haber diferencias de separador decimal según la configuración regional; para esta prueba interesa identificar los productos y su cantidad.

Qué cambió: hemos activado la presentación. No hemos añadido productos al controlador ni cambiado la URL. La vista usa los valores existentes en la propiedad `Model`.

Observación adicional: pulsa F12 → Red/Network, recarga `/Productos` y examina el HTML de respuesta. Busca `id="listado-productos"`. El navegador ha recibido elementos `<li>`; no recibe objetos C# ni ejecuta `@foreach`.

Detener el proceso antes del siguiente paso.

Profundización aplicada: Distinguir el valor del modelo y la plantilla de presentación.

Activar solo el envío de datos.

Partimos de la versión inicial, con el bloque del controlador aún comentado. Retira las dos líneas delimitadoras del comentario `PASO 1.2.3` y comenta la instrucción `return View();` que se utilizaba antes. No cambies todavía la vista.

Compila y ejecuta:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/Productos`. Debe aparecer un mensaje indicando tres productos, pero no las filas de Cuaderno, Lápiz y Carpeta.

Diagnóstico: el dato de la colección ha llegado correctamente al archivo de vista; lo sabemos porque `@Model.Count` puede obtener su cantidad. La ausencia de filas no demuestra que falte el modelo. Demuestra que todavía no se ha activado su representación detallada.

Activar el bucle de representación.

Detén el proceso y abre `Views/Productos/Index.cshtml`. Localiza el bloque `PASO 1.2.4`:

```
@* PASO 1.2.4 (inicio)
<ul id="listado-productos">
@foreach (var producto in Model)
{
    <li data-id="@producto.Id">@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
}
</ul>
PASO 1.2.4 (fin) *@
```

Elimina únicamente las dos líneas de comentario Razor. Guarda, compila y ejecuta. En `/Productos` deben aparecer los tres productos. El código del controlador no ha cambiado desde el estado anterior.

Comprueba que cada elemento tiene un atributo `data-id` que coincide con su identificador. No basta con ver los nombres: identifica el valor `1` junto a Cuaderno, `2` junto a Lápiz y `3` junto a Carpeta en el HTML.

Seguir una sola propiedad desde su origen hasta la respuesta.

Elige el producto Lápiz. Localiza `Nombre = "Lápiz"` dentro del inicializador del controlador. Comprueba que `Producto` tiene la propiedad `Nombre` y que la vista lee `producto.Nombre`.

El recorrido argumentado es:

1. La clase `Producto` define que existe una propiedad `Nombre`.
2. El controlador crea un objeto y establece su valor en `"Lápiz"`.
3. La colección de productos se entrega a la vista mediante `View(productos)`.
4. El `foreach` extrae cada objeto de la colección.
5. La expresión `@producto.Nombre` representa el valor recibido.
6. El navegador muestra el texto dentro de un elemento `<li>`.

Repite la explicación para `Id` y `Precio`. La primera se representa como atributo HTML; la segunda se formatea con dos decimales. Los mismos objetos dan lugar a diferentes partes del marcado según las expresiones de la vista.

Comprobar que el navegador no recibe el modelo C# original.

En F12 → Red, recarga `/Productos` y consulta el cuerpo de la respuesta. Busca la cadena `listado-productos` y observa los tres elementos `<li>`.

No debes encontrar el código ejecutable `new List<Producto>` dentro de la respuesta, porque la colección se construye en el servidor. Tampoco se envía al navegador el bucle `@foreach` para que lo ejecute: Razor ya ha representado su resultado.

Esto demuestra la diferencia entre el archivo fuente `.cshtml` y el HTML resultante. Es una comprobación sencilla que se volverá importante al comparar las vistas MVC con los componentes Blazor del tercer día.

Paso 1.2.5. Activar navegación reutilizando el layout

Abrir `Views/Shared/_Layout.cshtml`, localizar el enlace activo Inicio y, justo después, el fragmento comentado:

```
<a href="/">Inicio</a>
@* PASO 1.2.5 (inicio)
<a href="/Productos">Productos</a>
PASO 1.2.5 (fin) *@
```

Retirar las dos líneas que delimitan el comentario Razor de **1.2.5**, conservando los dos enlaces. No cambiar `@RenderBody()` ni el resto del layout.

Reiniciar la aplicación, abrir `/` y comprobar que la cabecera incluye los enlaces **Inicio** y **Productos**. Pulsar Productos; debe mostrarse la misma lista de tres productos que acabamos de activar. Pulsar Inicio para regresar al contenido de 1.1.

Qué hemos aprendido: cambiar el menú no obliga a tocar la clase `Producto` ni el controlador. El layout es una pieza de presentación reutilizable. En el capítulo 1.5 profundizaremos en layouts y vistas parciales; aquí interesa únicamente la separación entre navegación, selección de datos y representación.

Comprobación extra: escribir `http://localhost:5091/Productos/Index` y confirmar que conduce a la misma página que `/Productos`. El funcionamiento se debe al enrutamiento convencional de `Program.cs`.

Profundización aplicada: El layout como responsabilidad distinta.

El paso 1.2.5 permite activar un enlace adicional en el diseño compartido. En `Views/Shared/_Layout.cshtml` se encuentra:

```
@* PASO 1.2.5 (inicio)
<a href="/Productos">Productos</a>
PASO 1.2.5 (fin) *@
```

Al retirar las dos líneas delimitadoras del comentario Razor, el enlace aparece en las páginas que utilizan el layout. No hace falta editar `Models/Producto.cs` ni cambiar el código de `ProductosController.Index()`.

Esta es una buena oportunidad para comparar tres modificaciones consecutivas:

Cambio	Archivo responsable	Evidencia
Entregar los productos	`ProductosController.cs`	Aparece el contador
Mostrar los productos	`Views/Productos/Index.cshtml`	Aparecen filas de lista
Mostrar acceso desde el menú	`Views/Shared/_Layout.cshtml`	Aparece el enlace Productos

Explica por qué los tres cambios no deberían combinarse en una sola modificación de código. El objetivo es mantener visible qué responsabilidad corresponde a cada archivo y poder corregir un fallo sin modificar piezas ajenas.

Relacionar el layout con la navegación del navegador

Cuando haces clic en Productos, el navegador solicita `/Productos`. El enlace HTML no envía directamente una lista C# al cliente. Simplemente contiene una dirección que el servidor debe atender.

Si el enlace existiera pero no hubiera un controlador compatible, el navegador podría llegar a una respuesta 404. Por eso una comprobación completa debe verificar tanto la presencia del enlace como la respuesta de la dirección a la que apunta.

Paso 1.2.6. Explicar el recorrido con el código abierto

Con la página Productos cargada, reproduce el recorrido usando los archivos del proyecto:

1. El navegador solicita `GET /Productos`.
2. La ruta convencional conduce a `ProductosController.Index()`.
3. La acción crea tres instancias de `Producto` y las reúne en una colección.
4. `return View(productos)` entrega la colección a la vista correspondiente.
5. La vista comprueba que `Model` no es `null`, lee `Model.Count` y recorre los elementos.
6. El layout rodea el resultado con navegación; el servidor envía el HTML.
7. El navegador interpreta el HTML y muestra la lista.

Resultado esperado: debes poder identificar el archivo que interviene en cada una de estas etapas. No es suficiente decir «lo hace MVC»: hay que diferenciar qué pieza hace qué trabajo.

Reto: aplicación de los pasos

Consolidación y reto 1.2 — modificar los datos sin reescribir la vista.

Enunciado.

Con todos los pasos 1.2.3, 1.2.4 y 1.2.5 activos y la página `/Productos` funcionando, cambia el primer producto de **Cuaderno** a **Bloc**, manteniendo `Id = 1`, `Precio = 4.50m` y la cantidad total de **tres** productos. No toques `Views/Productos/Index.cshtml`.

Pregunta que debes resolver: si cambia el nombre en el controlador pero no se modifica el bucle `@foreach` de la vista, ¿por qué cambia automáticamente el texto mostrado al usuario?

Código preparado dentro del proyecto.

Abre `M01_MVC/PROYECTO/1.2/CursoWeb.Mvc/Controllers/ProductosController.cs`. Dentro de la lista del **paso 1.2.3**, una vez retirados los delimitadores de comentario de dicho paso, encontrarás las siguientes líneas:

```
new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
// RETO 1.2 — Comentar la línea de Cuaderno y descomentar la siguiente:
// new() { Id = 1, Nombre = "Bloc", Precio = 4.50m },
new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
```

Acción 1: comenta únicamente la primera declaración de producto (Cuaderno), añadiendo `//` delante.
Acción 2: elimina las dos barras `//` delante de la declaración de Bloc. No modifiques las restantes.

Así queda la parte relevante de la solución, con la variante anterior conservada como referencia:

```
// new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
// RETO 1.2 — Comentar la línea de Cuaderno y descomentar la siguiente:
new() { Id = 1, Nombre = "Bloc", Precio = 4.50m },
new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
```

Por qué funciona: la lista `productos` sigue conteniendo tres instancias de `Producto`. La primera tiene un valor diferente para `Nombre`; la vista simplemente accede a `producto.Nombre` en su bucle y, por eso, mostrará «Bloc». No hay lógica de negocio nueva ni se ha duplicado la definición de la vista.

Ejecutar y comprobar el reto.

Desde el directorio `M01_MVC/PROYECTO/1.2/CursoWeb.Mvc`:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/Productos`. Debes obtener **Bloc, Lápiz y Carpeta**, conservando los precios y el contador de tres productos. Comprueba que ya no aparece «Cuaderno» en la lista. La ruta alternativa `/Productos/Index` debe mostrar exactamente lo mismo.

Prueba automatizada del reto: desde la raíz del repositorio, `bash M01_MVC/TESTS/1.2_e2e.sh`. El escenario `reto` reconstruye una copia temporal del proyecto con el listado y la navegación activados, cambia la línea de Cuaderno por la de Bloc, compila y verifica realmente la respuesta HTTP y sus tres productos. El estado inicial del alumno no se modifica.

Preguntas y solución razonada.

¿Qué archivo cambiarías para mostrar los nombres en una etiqueta HTML diferente? `Views/Productos/Index.cshtml`, ya que decide la presentación.

¿Qué cambiarías para modificar el precio inicial? El valor de `Precio` del objeto en la acción `ProductosController.Index()`; la vista lo consume con `producto.Precio`.

¿Por qué el cambio se pierde si se revierte el código o se reinicia sin recompilarlo? Los datos del catálogo son creados en memoria por el método; no son registros de una base de datos. La persistencia se introducirá en el día 2.

Conclusión del reto: se han utilizado **dos variantes de código ya presentes** en el proyecto; se ha ejecutado el nuevo estado sin inventar piezas ausentes y se ha demostrado la separación entre suministro de datos y representación.

Resolver el reto con una evidencia más estricta.

El reto 1.2 pide sustituir **Cuaderno** por **Bloc** sin cambiar el archivo Razor de la vista. El fragmento preparado en el controlador conserva las dos variantes para que puedas alternarlas:

```
new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
// RETO 1.2 — Comentar la línea de Cuaderno y descomentar la siguiente:
// new() { Id = 1, Nombre = "Bloc", Precio = 4.50m },
```

Comenta la primera declaración y descomenta la segunda. No actives ambas a la vez, porque el objetivo no es añadir un producto sino reemplazar el nombre de uno existente. Deben seguir existiendo tres registros.

Después de compilar y reiniciar, examina `/Productos`. Busca Bloc y confirma que el contador sigue mostrando tres. Comprueba también que Lápiz y Carpeta conservan su nombre y precio. Este último paso evita una falsa solución que cambie el nombre pero también altere accidentalmente el resto del catálogo.

Comprobación detallada de la solución

En las herramientas de desarrollo revisa el HTML de la lista. El elemento con `data-id="1"` debe contener Bloc. Los elementos `data-id="2"` y `data-id="3"` deben conservar Lápiz y Carpeta.

No modifiques el bloque `@foreach` para escribir literalmente Bloc. Una solución así produciría un resultado visual parecido, pero incumpliría el objetivo de la práctica: comprobar que la vista representa el valor recibido desde el modelo.

Pregunta de razonamiento: ¿qué ocurriría si cambiáramos el valor en la vista pero dejáramos los datos originales en el controlador? Habríamos introducido una discrepancia entre el modelo y su representación. Cuando otra pantalla consumiera el mismo modelo, podría seguir mostrando Cuaderno. Por eso conviene que la vista consulte el dato en lugar de inventarlo.

El script `1.2_e2e.sh` reconstruye el estado de reto sobre una copia temporal y comprueba la sustitución. También verifica que el catálogo sigue teniendo tres elementos y que las rutas anteriores continúan funcionando.

Comprobaciones funcionales y casos de diagnóstico

Resultado final y ejecución.

Con los tres bloques habilitados, el proyecto compila y `http://localhost:5091/Productos` muestra:

```
Inicio   Productos

1.2 | Modelo, Vista y Controlador
La vista decide cómo presentar los datos, pero no inventa el catálogo.

El controlador ha enviado 3 productos.

• Cuaderno — 4.50 €
• Lápiz — 1.20 €
• Carpeta — 3.75 €
```

Los textos decimales pueden adoptar el separador regional del proceso. El HTML real incluye el layout, estilos y atributos adicionales. Lo importante es comprobar que el recuento y los tres registros proceden de la colección entregada a la vista.

La página inicial `/` continúa funcionando. La ruta `/Productos/Index` es equivalente a `/Productos`. Un recurso desconocido debe seguir respondiendo 404.

Comprobación funcional por estados.

Estado probado	Cambios activados	Resultado observable
Inicial	Ninguno	`/Productos` devuelve 200 y muestra «todavía no ha entregado una lista»
Entrega de datos	1.2.3	Aparece el contador de 3 productos, pero no la lista
Vista activada	1.2.3 + 1.2.4	Aparecen contador y tres productos; aún no existe el enlace adicional

Estado probado	Cambios activados	Resultado observable
Navegación activada	1.2.3 + 1.2.4 + 1.2.5	Se añade el enlace Productos y permite navegar al catálogo

Para realizar una comprobación automatizada, desde la **raíz del repositorio**:

```
bash M01_MVC/TESTS/1.2_e2e.sh
```

Este script construye el proyecto en su estado inicial y activa las demás combinaciones mediante copias temporales. Arranca realmente Kestrel y verifica las respuestas HTTP, los textos visibles en el HTML, las rutas, los recursos estáticos y la permanencia de la página de 1.1. Requiere SDK .NET 8, Bash, Python 3 y curl; se utiliza también en GitHub Actions. No sustituye la observación de la interfaz con un navegador durante la clase.

Criterio de finalización del ejercicio: puedes explicar por qué el listado no aparece en el estado inicial, por qué el contador aparece antes de activar `@foreach` y por qué el enlace adicional no requiere cambiar el controlador.

Resolución de incidencias

Problemas previsibles y soluciones.

Error de compilación por código inalcanzable o varias instrucciones de retorno. En 1.2.3 se olvidó comentar la instrucción `return View();` original. Debe quedar solamente una ruta de retorno activa en la acción, la que ejecuta `View(productos)`.

La página sigue diciendo que no recibió modelo. Comprobar que los delimitadores `/* ... */` del paso 1.2.3 se quitaron, que la instrucción `return View(productos);` está activa y que se ha reiniciado la aplicación.

El contador muestra tres, pero no hay lista. Todavía no se ha activado el bloque `@foreach` de 1.2.4. Es un estado didáctico válido, no un error del controlador.

Se obtiene un error Razor al activar la lista. Comprobar que se quitaron ambas líneas delimitadoras `@* ... *@` y que permanecen bien emparejadas las llaves del `foreach` y las etiquetas `<ul>` y `<li>`.

Aparece el listado pero no el enlace Productos. El menú depende de `Views/Shared/_Layout.cshtml`, no de la vista `Views/Productos/Index.cshtml`. Activar el bloque 1.2.5 y recargar desde la página inicial.

El navegador abre un proyecto anterior. La carpeta del capítulo 1.1 tiene el mismo nombre de proyecto, porque es una versión conservada; hay que arrancar explícitamente `PROYECTO/1.2/CursoWeb.Mvc`. No ejecutar ambas copias sobre el mismo puerto.

Las cantidades monetarias usan otro separador. `ToString("0.00")` utiliza la configuración regional del proceso cuando no se especifica una cultura. La finalidad de este ejercicio es recorrer datos del modelo, no configurar internacionalización.

Cambiar la clase `Producto` no modifica mágicamente los valores ya enviados. La clase define qué atributos existen; son los objetos creados en la acción los que contienen los valores concretos del catálogo. Modificar propiedades de un tipo y modificar las instancias son operaciones diferentes.

Un 404 en `/Productos`. Comprobar que `ProductosController.cs` está en `Controllers`, que existe `Index()` y que el programa registra la ruta convencional. No confundir esta URL con `/api/productos`, que no existe en nuestra aplicación MVC.

Diagnóstico de errores de modelo y vista.

El contador está presente pero no hay elementos.

Es el estado normal después de activar 1.2.3 y antes de activar 1.2.4. No cambies el controlador por segunda vez; revisa el bloque comentado en la vista.

Aparece el mensaje de ausencia de modelo.

La vista sigue recibiendo un modelo nulo. Comprueba que el bloque `return View(productos);` se ha activado correctamente y que la instrucción anterior `return View();` no ha quedado como retorno utilizado para la acción.

El proyecto deja de compilar.

Si aparecen errores después de descomentar el controlador, verifica las dos líneas que delimitaban el comentario y la posición de `return View();`. No conviene eliminar llaves ni reescribir el método entero sin localizar primero qué instrucción ha quedado activa.

Si el error surge al descomentar Razor, comprueba que has quitado las dos líneas `@* ... *@` y que el `foreach` permanece dentro del bloque `else` de la comprobación de modelo no nulo.

Las dos direcciones de catálogo no muestran lo mismo.

Comprueba que sigues utilizando la ruta convencional y que ambas direcciones seleccionan `ProductosController.Index()`. No hay que duplicar la vista para cada URL. El código de la acción y la convención de búsqueda son suficientes en el proyecto actual.

La página de inicio ha cambiado inesperadamente.

El objetivo de 1.2 no es modificar la portada del punto 1.1. Si desaparece, revisa si has editado `HomeController` o el archivo `Views/Home/Index.cshtml` por error, o si has abierto otra versión del proyecto.

Resultado y continuidad

Continuidad hacia 1.3.

Conservar el resultado de este capítulo. En `1.3` estudiaremos detalladamente las carpetas y convenciones que acabamos de recorrer y prepararemos la organización del proyecto para ampliar más adelante las acciones y las vistas.

No añadir ahora formularios, EF Core, autenticación ni servicios externos. En este capítulo se ha demostrado una arquitectura MVC mínima, suficiente para visualizar y comprender el paso de objetos del controlador a la vista.

Consolidación guiada para cerrar el flujo MVC.

Toma como referencia la página final de productos y el árbol de archivos. Sin ejecutar nada durante unos minutos, describe el recorrido completo de una solicitud `GET /Productos` utilizando los nombres reales de los tipos y archivos.

Después vuelve a abrirlos para comprobar tu explicación. Debes poder identificar la ruta convencional, la acción `Index()`, el tipo `Producto`, la instrucción `View(productos)`, la directiva `@model` y el bucle `@foreach`.

Realiza una última comprobación del reto Bloc y explica por qué se modifica el dato en el controlador y no el texto HTML de la vista. El resultado correcto conserva tres productos, la portada anterior y la navegación.

La finalidad de esta ampliación no es memorizar más nombres, sino reforzar una relación causal: **un cambio en los datos entregados provoca un cambio de contenido; un cambio de plantilla provoca un cambio de representación; un cambio en el layout modifica la estructura compartida de navegación**.

Al finalizar, la siguiente etapa podrá dedicarse a la estructura del proyecto, los archivos de configuración y la publicación de recursos estáticos, sin necesidad de volver a explicar la separación entre modelo, vista y controlador.

P1.3. Estructura y convenciones de un proyecto ASP.NET Core MVC

Objetivo y preparación

Objetivo práctico: identificar dónde se encuentra cada parte de una aplicación MVC, comprender la diferencia entre arranque y atención de peticiones, leer un valor de configuración, añadir recursos estáticos y comprobar qué rutas son públicas mediante `UseStaticFiles()`. El reto final introduce una segunda URL de texto accesible, cuyo enlace se activará utilizando el **código que ya se encuentra en el proyecto**.

Teoría asociada: apartado 1.3 completo de `M01_TEORIA.md`. Se utilizarán los conceptos ya estudiados de HTTP y MVC de 1.1 y 1.2.

Estado de partida: proyecto del capítulo **1.3**, que conserva y deja activos los pasos funcionales 1.2.3, 1.2.4 y 1.2.5. La página de productos puede abrirse desde el principio con tres registros. No es necesario repetir el desbloqueo del controlador y la vista del capítulo anterior.

Duración de aula prevista: 20 minutos de explicación y 35 minutos de práctica; la extensión de ejemplos y comprobaciones del documento permite repasar o completar observaciones fuera de ese tiempo. Las comprobaciones orientadas a docentes se automatizan para no consumir tiempo lectivo de los alumnos.

Necesitamos **SDK .NET 8**, Visual Studio con soporte ASP.NET Core o un editor compatible con C#, navegador web con herramientas de desarrollo y terminal. En este punto no se usa base de datos, API externa ni Docker.

Desde una copia del repositorio y posicionándose **en su raíz**:

```
cd M01_MVC/PROYECTO/1.3/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/`. Debe aparecer la portada ya conocida. Pulsar `Productos` en la navegación. La dirección `http://localhost:5091/Productos` debe mostrar **Cuaderno, Lápiz y Carpeta**, con el contador de tres productos.

Observación inicial: el enlace a productos y la vista de productos están activos porque son la base funcional de 1.3. No es una aplicación nueva; es la continuación del mismo ejemplo didáctico, guardada en una carpeta autónoma para que todas las etapas permanezcan disponibles.

Detener el servidor con **Ctrl+C** y mantener abierta la carpeta del proyecto. Los cambios de código se probarán reiniciándolo para identificar sin ambigüedad el estado ejecutado.

Mapa verificable de cada paso

Cada fila permite localizar exactamente qué debe consultar o modificar el alumno. **Observar** significa que el código ya está activo y no debe descomentarse; **activar** significa que el proyecto incluye el bloque preparado. Las pruebas E2E ejecutan los estados indicados, pero no reemplazan la observación manual del navegador.

Paso	Archivo y acción del alumno	Resultado y comprobación
1.3.1	`CursoWeb.Mvc.csproj` ; `Program.cs` . **Acción:** Reconocer framework y árbol de proyecto	**Resultado:** Compilación `.NET 8` . **Prueba:** Compilación de `1.3_e2e.sh`
1.3.2	`Program.cs` ; `appsettings.json` . **Acción:** Identificar pipeline y recursos sin alterarlos	**Resultado:** Catálogo y portada continúan funcionando. **Prueba:** Estado inicial de `1.3_e2e.sh`
1.3.3	`Program.cs` ; `appsettings.json` . **Acción:** Activar registro `app.Configuration["Curso:Nombre"]`	**Resultado:** Mensaje de configuración en el log. **Prueba:** Estado de configuración de `1.3_e2e.sh`
1.3.4	`Views/Shared/_Layout.cshtml` ; `wwwroot/css/catalogo.css` . **Acción:** Activar el enlace CSS preparado	**Resultado:** CSS disponible con HTTP 200 y enlace en HTML. **Prueba:** Estado CSS de `1.3_e2e.sh`
1.3.5	`Views/Shared/_Layout.cshtml` ; `wwwroot/recursos/guia-publica.txt` . **Acción:** Activar enlace al recurso público	**Resultado:** GET `/recursos/guia-publica.txt` → 200. **Prueba:** Estado de recurso de `1.3_e2e.sh`
Reto 1.3	`Views/Shared/_Layout.cshtml` ; `DOCUMENTACION/nota-interna.txt` . **Acción:** Activar enlace de aviso; verificar que el interno no se publica	**Resultado:** Aviso 200; documento fuera de `wwwroot` → 404. **Prueba:** Escenario de reto de `1.3_e2e.sh`

Regla de trazabilidad. No se considera resuelto un reto solo porque se vea su nombre en el código: se comprueba el resultado esperado con el proyecto activado. Los bloques comentados permanecen como material de ejecución del alumno; no deben sustituirse por capturas ni pseudocódigo.

Desarrollo práctico paso a paso

Paso 1.3.1 — Inspeccionar la estructura real del proyecto

Desde el explorador de soluciones de Visual Studio o desde el explorador de archivos, identificar:

```
M01_MVC/PROYECTO/1.3/CursoWeb.Mvc/
├── CursoWeb.Mvc.csproj
├── Program.cs
├── appsettings.json
├── Properties/launchSettings.json
├── Controllers/HomeController.cs
├── Controllers/ProductosController.cs
├── Models/Producto.cs
├── Views/Home/Index.cshtml
├── Views/Productos/Index.cshtml
├── Views/Shared/_Layout.cshtml
├── Views/_ViewStart.cshtml
├── Views/_ViewImports.cshtml
├── wwwroot/css/site.css
├── wwwroot/css/catalogo.css
├── wwwroot/recursos/guia-publica.txt
├── wwwroot/recursos/aviso-laboratorio.txt
└── DOCUMENTACION/nota-interna.txt
```

No modificar todavía ningún archivo. El objetivo es asociar cada necesidad a una ubicación razonable.

Completar verbalmente las siguientes relaciones:

Necesidad	Archivo o carpeta donde buscar
Cambiar el tipo de proyecto o framework	`CursoWeb.Mvc.csproj`
Comprender qué se registra al arrancar	`Program.cs`
Mostrar otro producto en la lista fija	`Controllers/ProductosController.cs`
Modificar el marcado HTML del listado	`Views/Productos/Index.cshtml`
Encontrar las propiedades de un producto	`Models/Producto.cs`
Añadir un enlace común a todas las páginas	`Views/Shared/_Layout.cshtml`
Añadir una hoja CSS pública	`wwwroot/css/`
Definir ajustes no sensibles del curso	`appsettings.json`

Por qué esta identificación es útil: al ampliar una aplicación, la mayor parte de los errores de mantenimiento comienzan por cambiar un archivo que no es responsable del comportamiento. Saber dónde mirar reduce modificaciones innecesarias y favorece la separación MVC.

Comprueba además que `bin/` y `obj/`, cuando aparecen tras compilar, son resultados generados; no forman parte del código del alumno que debe versionarse.

Profundización aplicada: Examinar el proyecto antes de modificarlo.

Elaborar un mapa funcional de archivos.

Abre la carpeta `M01_MVC/PROYECTO/1.3/CursoWeb.Mvc` y localiza estos archivos del proyecto:

```
CursoWeb.Mvc.csproj
Program.cs
appsettings.json
Controllers/HomeController.cs
Controllers/ProductosController.cs
Models/Producto.cs
Views/Productos/Index.cshtml
Views/Shared/_Layout.cshtml
Views/_ViewStart.cshtml
Views/_ViewImports.cshtml
wwwroot/css/catalogo.css
wwwroot/recursos/guia-publica.txt
DOCUMENTACION/nota-interna.txt
```

Sin modificar nada, responde para cada archivo si pertenece principalmente a configuración, arranque, acción de controlador, modelo de datos, presentación Razor, recurso estático o documentación interna. Algunas carpetas pueden intervenir en más de un proceso, pero cada archivo debe tener una responsabilidad principal identificable.

En tus apuntes prepara una tabla de tres columnas: archivo, función y evidencia concreta. Por ejemplo, para `Program.cs` la evidencia puede ser la llamada `builder.Services.AddControllersWithViews()` o `app.MapControllerRoute(...)`. Para la vista de productos, la evidencia puede ser `@model IReadOnlyList<...>`. Evita responder solo «es un archivo importante»: debes asociarlo a una instrucción.

Leer el archivo de proyecto sin confundirlo con Program.cs.

Abre `CursoWeb.Mvc.csproj` e identifica:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
```

```
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Pregunta 1: ¿qué framework utiliza el proyecto? `net8.0`. **Pregunta 2:** ¿se ejecuta Kestrel simplemente por abrir este archivo? No; el archivo describe el proyecto que debe construirse. **Pregunta 3:** ¿dónde se organiza el arranque de la aplicación? En `Program.cs`.

Ejecuta:

```
dotnet restore
dotnet build
```

Revisa el resultado de compilación. El proyecto contiene dependencias de la plataforma web, pero no necesita añadir EF Core para completar este apartado. Si falla una operación, distingue primero si se trata de restauración, compilación o ejecución.

Identificar bin y obj después de compilar.

Tras construir el proyecto, revisa las carpetas generadas `bin` y `obj`. No es necesario modificar su contenido; de hecho, el objetivo del ejercicio es comprender por qué no deben tratarse como archivos fuente que el alumno tenga que editar.

Compara el estado del árbol antes y después de compilar. Las clases C# y las vistas Razor siguen estando en sus carpetas originales, mientras que el SDK puede haber creado archivos auxiliares y ensamblados. Esta observación ayudará en prácticas posteriores, donde se compilarán varias versiones del mismo proyecto.

Paso 1.3.2 — Entender Program.cs antes de tocar una página

Abrir `Program.cs` e identificar el siguiente recorrido:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Se omite aquí únicamente el bloque opcional del paso 1.3.3.
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

El fragmento representa las instrucciones activas y esenciales. El archivo completo contiene también comentarios docentes.

Explicación de cada instrucción:

1. `CreateBuilder(args)` prepara la aplicación y las fuentes de configuración.
2. `AddControllersWithViews()` registra los servicios de MVC.
3. `Build()` construye el host.
4. `UseStaticFiles()` permite enviar CSS y texto desde `wwwroot` cuando sus tipos son reconocidos.
5. `UseRouting()` participa en el enrutamiento.

6. `MapControllerRoute` registra una convención que permite llegar a `HomeController.Index` y `ProductosController.Index`.
7. `Run()` arranca el servidor. La acción de productos se ejecutará cuando llegue una petición, no por el mero hecho de registrar la ruta.

Comprobación: sin añadir ningún código, abre `/Productos` y `/css/site.css` y observa que ambos devuelven HTTP 200, pero el primero produce HTML MVC y el segundo CSS. Se atienden solicitudes de naturaleza distinta dentro de una misma aplicación ASP.NET Core.

Paso 1.3.3 — Leer un valor desde appsettings.json

Primero abrir `appsettings.json` y localizar:

```
"Curso": {
  "Nombre": "Curso MVC de productos",
  "Edicion": "Laboratorio NET8"
}
```

Este fragmento forma parte de un JSON completo que ya se encuentra correctamente configurado. No se deben crear comentarios de estilo C# dentro de `appsettings.json`, porque el ejercicio no los necesita y podrían volver inválido el archivo.

Acción: abrir `Program.cs` y localizar la instrucción comentada con la marca `PASO 1.3.3`:

```
// app.Logger.LogInformation("P1.3 configuracion: {Nombre}", app.Configuration["Curso:Nombre"]);
```

Quitar **solo** las dos barras `//` delante de la instrucción. No eliminar el resto de la configuración. La línea activa queda:

```
app.Logger.LogInformation("P1.3 configuracion: {Nombre}", app.Configuration["Curso:Nombre"]);
```

Cómo funciona: `app.Configuration["Curso:Nombre"]` obtiene el valor situado en la sección `Curso`, clave `Nombre`, cargada desde la configuración. `app.Logger.LogInformation` emite un mensaje de nivel *Information* en el registro del proceso; `{Nombre}` es un marcador de valor del sistema de logging.

Construir y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

En la consola de ejecución debe aparecer una entrada equivalente a:

```
P1.3 configuracion: Curso MVC de productos
```

La estructura y los prefijos de fecha o categoría del registro pueden variar; el contenido es lo relevante.

Resultado esperado: la aplicación sigue mostrando sus dos páginas anteriores; la instrucción nueva escribe **en el registro de la aplicación**, no en el HTML. Se demuestra así que un ajuste de `appsettings.json` puede leerse durante el arranque sin tocar una vista.

Detener el proceso antes del siguiente paso.

Profundización aplicada: Relacionar configuración y ejecución.

Identificar la sección de configuración.

Abre `appsettings.json`. Localiza la sección `Curso`:

```
"Curso": {
  "Nombre": "Curso MVC de productos",
  "Edicion": "Laboratorio NET8"
}
```

No es necesario añadir comentarios JSON ni cambiar el nombre de las propiedades. El proyecto suministrado ya contiene los valores adecuados para observar el acceso a configuración.

En `Program.cs`, encuentra la instrucción de registro del paso 1.3.3:

```
// app.Logger.LogInformation("P1.3 configuracion: {Nombre}", app.Configuration["Curso:Nombre"]);
```

Elimina las dos barras iniciales y deja la instrucción activa. El código accede a una clave jerárquica y utiliza su valor para escribir un mensaje en el log de aplicación.

Contrastar la consola y el navegador.

Compila, inicia la aplicación y observa la terminal:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Debes localizar el mensaje `P1.3 configuracion: Curso MVC de productos`. A continuación abre `/Productos` en el navegador y verifica que el catálogo continúa mostrando sus tres registros.

Pregunta: ¿por qué el texto de `Curso:Nombre` aparece en la consola pero no como un nuevo producto en la lista? Porque el código del ejercicio solo registra ese valor en el host. No hemos modificado la colección del controlador ni la vista para representar la configuración como contenido de página.

Este contraste ayuda a separar los datos de configuración y los datos del modelo `Producto`.

Comparar la URL de ejecución con los valores del JSON.

El comando utilizado especifica `http://localhost:5091`. La sección `Curso` no contiene el puerto ni configura el servidor. Si un alumno interpreta que el valor `"Laboratorio NET8"` controla la dirección de escucha, deberá revisar el argumento `--urls` y la función de `appsettings.json` en el ejemplo.

No es necesario investigar todos los proveedores de configuración en esta jornada. El objetivo es demostrar que un valor puede leerse desde una sección JSON sin mezclarlo con el código que obtiene productos.

Paso 1.3.4 — Incorporar una hoja CSS sin crear una acción MVC

La carpeta `wwwroot/css` incluye `catalogo.css`, cuyo contenido ya está activo:

```
/* PASO 1.3.4: recurso estático independiente de los controladores. */
/* La regla está activa en este archivo; el alumno activa su enlace en _Layout.cshtml. */
main.container {
    border-left: 5px solid #2d635d;
    padding-left: 1.5rem;
}
```

Qué hace: cambia la presentación del elemento HTML `<main class="container">`, añadiendo un borde izquierdo y espacio interior. No contiene Razor ni código C#, y por ello el controlador no necesita modificarse.

Acción: abrir `Views/Shared/_Layout.cshtml`. Bajo la referencia a `site.css`, localizar:

```
@* PASO 1.3.4 (inicio)
<link rel="stylesheet" href="/css/catalogo.css" />
PASO 1.3.4 (fin) *@
```

Eliminar las **dos líneas** que delimitan ese comentario Razor. Dejar activa exactamente la siguiente etiqueta, manteniendo la hoja anterior:

```
<link rel="stylesheet" href="~/css/site.css" />
<link rel="stylesheet" href="/css/catalogo.css" />
```

Por qué es necesario: el archivo `catalogo.css` ya existe físicamente en `wwwroot` y puede descargarse si se conoce su URL, pero el navegador solo aplicará sus reglas a la página cuando el HTML recibido incluya una etiqueta `<link rel="stylesheet">` adecuada.

Reiniciar la aplicación y abrir `http://localhost:5091/Productos`. Debe apreciarse un **borde vertical** en el área principal. En F12 → Red, comprobar que además de la hoja `site.css` el navegador solicita `/css/catalogo.css` y recibe HTTP 200 y tipo `text/css`.

Prueba independiente: abre `http://localhost:5091/css/catalogo.css` directamente. Se verá el contenido CSS como respuesta HTTP. Esa URL no pasa por un controlador MVC.

Detener el servidor antes de seguir.

Profundización aplicada: Comprender el CSS como recurso HTTP.

Probar un archivo directamente antes de enlazarlo.

Sin activar todavía el paso 1.3.4, inicia el proyecto y abre en el navegador:

```
http://localhost:5091/css/catalogo.css
```

El recurso debe responder y mostrar el contenido CSS. En otra terminal puedes comprobarlo con:

```
curl -i http://localhost:5091/css/catalogo.css
```

Observa el estado 200 y el tipo de contenido CSS. Es importante realizar esta comprobación **antes de activar el enlace en el layout**: así demostramos que la publicación del archivo no depende de que exista una etiqueta `<link>` en la página de catálogo.

El CSS suministrado contiene:

```
main.container {
    border-left: 5px solid #2d635d;
    padding-left: 1.5rem;
}
```

Esta regla modifica la presentación del área principal cuando el navegador carga la hoja. No selecciona el controlador, no construye productos y no altera la URL `/Productos`.

Habilitar el enlace y observar una segunda solicitud.

Abre `Views/Shared/_Layout.cshtml` y localiza el bloque `PASO 1.3.4`:

```
@* PASO 1.3.4 (inicio)
<link rel="stylesheet" href="/css/catalogo.css" />
PASO 1.3.4 (fin) *@
```

Elimina las líneas delimitadoras del comentario Razor. Compila y ejecuta de nuevo, abre `/Productos` y consulta F12 → Red.

Además del documento principal, el navegador deberá solicitar `/css/catalogo.css`. Comprueba el tipo de contenido y el estado. Observa también el borde izquierdo del área principal, que depende de la regla CSS.

Distinguir un fallo HTTP de uno visual.

Si el borde no aparece, no concluyas inmediatamente que el CSS falta. Primero comprueba si el archivo se descargó correctamente. Si el recurso devuelve 200, revisa el enlace y las reglas aplicadas al elemento `main.container` mediante el inspector. Si devuelve 404, revisa la ruta y la ubicación física del archivo.

La decisión de diagnóstico depende de una evidencia previa. Un problema de carga HTTP y un problema de prioridad entre reglas CSS requieren comprobar elementos distintos.

Paso 1.3.5 — Enlazar un recurso de texto publicado desde wwwroot

Dentro de `wwwroot/recursos/` existe el archivo `guia-publica.txt`, cuyo contenido es:

```
PASO 1.3.5 - Guia publica del curso ASP.NET Core MVC.
Este archivo se sirve desde wwwroot mediante UseStaticFiles.
No es necesario crear un controlador para atender esta URL.
```

Acción: abrir `Views/Shared/_Layout.cshtml` y localizar la marca `PASO 1.3.5`, junto a la navegación:

```
@* PASO 1.3.5 (inicio)
<a href="/recursos/guia-publica.txt">Guía pública</a>
PASO 1.3.5 (fin) *@
```

Retirar únicamente los dos delimitadores del comentario Razor. El enlace queda activo:

```
<a href="/recursos/guia-publica.txt">Guía pública</a>
```

Reiniciar la aplicación, abrir `/Productos` y comprobar que ahora hay un enlace **Guía pública** en la navegación. Pulsarlo: el navegador debe pedir `/recursos/guia-publica.txt` y mostrar el texto del archivo, no una vista Razor. Revisar en Red que la solicitud obtiene HTTP 200.

Observación decisiva: prueba también la URL directa `/recursos/guia-publica.txt` antes de descomentar el enlace, si deseas comparar estados. El recurso ya era accesible. Habilitar el enlace facilita encontrarlo en la interfaz, pero **no cambia la configuración de publicación estática**.

Aquí no hemos creado ninguna clase nueva ni un método de controlador. El trabajo ya lo hace el middleware de archivos estáticos.

Profundización aplicada: Archivos de texto públicos y no públicos.

Comparar las rutas físicas.

Abre estos dos archivos dentro del proyecto:

```
wwwroot/recursos/guia-publica.txt
DOCUMENTACION/nota-interna.txt
```

Ambos son textos y ambos forman parte del árbol del proyecto, pero solo el primero está dentro de la raíz web utilizada por el middleware de archivos estáticos.

Con la aplicación ejecutándose, prueba:

```
curl -i http://localhost:5091/recursos/guia-publica.txt
curl -i http://localhost:5091/DOCUMENTACION/nota-interna.txt
```

La primera URL debe devolver 200 y el contenido de la guía. La segunda debe devolver 404, porque nuestro proyecto no ha configurado una ruta que publique automáticamente la carpeta `DOCUMENTACION`.

Habilitar el enlace de la guía.

En el layout encuentra el bloque `PASO 1.3.5` y retira los delimitadores de comentario:

```
@* PASO 1.3.5 (inicio)
<a href="/recursos/guia-publica.txt">Guía pública</a>
PASO 1.3.5 (fin) *@
```

Tras activarlo, el enlace aparecerá en la navegación del sitio. Pulsa el enlace y confirma que llega al mismo archivo que ya habías probado mediante su URL directa.

Pregunta clave: ¿ha cambiado la posibilidad de acceder al archivo o únicamente la navegación? En este ejemplo ha cambiado la navegación. El archivo de `wwwroot` ya era público antes de añadir el enlace.

Evitar conclusiones incorrectas sobre seguridad.

No debemos interpretar la respuesta 404 de `nota-interna.txt` como una garantía universal de secreto. El archivo existe en el repositorio, y si el repositorio es público puede leerse desde el servicio donde está publicado el código fuente.

El resultado 404 solo muestra que el middleware estático de esta aplicación no publica directamente esa ruta con la configuración actual. Otra configuración de servidor o una acción específica podría exponerla. Este detalle es importante para no convertir una convención de carpetas en una supuesta política de protección de datos.

Reto: aplicación de los pasos

Reto 1.3 — Recurso público frente a archivo interno.

Enunciado

Se pide añadir un enlace visible llamado **Aviso de laboratorio**, que permita al alumno abrir un archivo de texto público. También debes localizar un segundo archivo de texto que **no se publique** por la configuración actual de `UseStaticFiles()` y explicar por qué produce otra respuesta.

El reto comprueba si sabes razonar a partir de la **estructura física del proyecto**, la raíz web y el layout, y no únicamente repetir los comandos del paso 1.3.5.

Código incluido en el proyecto

No necesitas crear archivos que falten; ya están incorporados en `PROYECTO/1.3/CursoWeb.Mvc`.

Archivo 1 — `wwwroot/recursos/aviso-laboratorio.txt`:

```
RETO 1.3 - Aviso de laboratorio.
El documento esta dentro de wwwroot y puede solicitarse como recurso estatico.
El enlace de navegacion del reto se activa por separado en _Layout.cshtml.
```

Archivo 2 — `DOCUMENTACION/nota-interna.txt`:

```
Nota didactica interna del proyecto (datos ficticios).
Al estar fuera de wwwroot, UseStaticFiles no la publica de forma predeterminada.
No colocar secretos reales en el repositorio: esta nota solo ilustra una convencion.
```

Archivo 3 — `Views/Shared/_Layout.cshtml`, con el siguiente bloque dentro del menú:

```
@* RETO 1.3 (inicio)
<a href="/recursos/aviso-laboratorio.txt">Aviso de laboratorio</a>
RETO 1.3 (fin) *@
```

Resolver y activar el reto

Acción 1: abre `Views/Shared/_Layout.cshtml`, localiza `RETO 1.3 (inicio)` y elimina **las dos líneas que delimitan el comentario**, sin cambiar el contenido del enlace. La solución activa queda:

```
<a href="/recursos/aviso-laboratorio.txt">Aviso de laboratorio</a>
```

Esta etiqueta HTML se añadirá al layout de ambas páginas. No se precisa modificar `ProductosController` ni `Program.cs`: el recurso ya está dentro de `wwwroot` y `UseStaticFiles()` ya forma parte del arranque.

Acción 2: compila y ejecuta de nuevo la aplicación:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/Productos` y pulsa el enlace Aviso de laboratorio. Debe mostrarse el contenido de `aviso-laboratorio.txt`. En la pestaña Red, comprobar el código **200**.

Acción 3: escribe directamente estas direcciones en el navegador o ejecútalas en una segunda terminal con `curl -i`:

```
http://localhost:5091/recursos/aviso-laboratorio.txt
http://localhost:5091/DOCUMENTACION/nota-interna.txt
```

La primera respuesta debe ser HTTP **200** y devolver el contenido del archivo público. La segunda debe devolver HTTP **404**, pues nuestro middleware no sirve por defecto archivos del directorio `DOCUMENTACION`.

Acción 4: explica por escrito por qué el enlace del reto no es un mecanismo de seguridad. La respuesta correcta es que un recurso bajo `wwwroot` **ya es accesible directamente** aunque no aparezca en la navegación. La ausencia del enlace no protege el contenido. Además, aunque `nota-interna.txt` no se sirva por el servidor web, forma parte de un repositorio público: este ejemplo no sirve para guardar secretos.

Solución explicada

El enlace del archivo de layout es la **única línea de HTML** que debe activarse para resolver el reto. Al hacerlo se modifica la navegación, pero la aplicación no necesita un endpoint controlador adicional porque ya conoce cómo servir archivos estáticos.

No cambiamos `wwwroot/recursos/aviso-laboratorio.txt`: el texto estaba disponible desde el inicio. Tampoco movemos `DOCUMENTACION/nota-interna.txt`, porque la comparación de ubicaciones es el objetivo didáctico.

La combinación de resultados confirma qué servicio está atendiendo cada petición:

```
GET /Productos                              -> 200, HTML MVC
GET /recursos/aviso-laboratorio.txt         -> 200, texto público
GET /DOCUMENTACION/nota-interna.txt        -> 404, no publicado como estático
```

Reto entregado con código: el Markdown describe la solución completa; el proyecto contiene los dos textos y el bloque del layout comentado; las pruebas automatizadas habilitan el fragmento y comprueban los tres resultados.

Resolver el reto y comprobar resultados negativos.

El reto del capítulo consiste en habilitar un enlace al archivo `aviso-laboratorio.txt`, que ya se encuentra en `wwwroot/recursos`.

En `Views/Shared/_Layout.cshtml` se proporciona:

```
@* RETO 1.3 (inicio)
<a href="/recursos/aviso-laboratorio.txt">Aviso de laboratorio</a>
RETO 1.3 (fin) *@
```

Retira las dos líneas de comentario Razor. No es necesario crear un controlador ni modificar `Program.cs`, porque el archivo ya está publicado como recurso estático.

Comprobar el acceso positivo.

Reinicia el proyecto, abre `/Productos` y pulsa Aviso de laboratorio. La URL `/recursos/aviso-laboratorio.txt` debe devolver 200 y el texto del recurso.

El hecho de que el enlace esté en el layout significa que puede aparecer tanto en la página inicial como en la página de productos. La ubicación del enlace no altera el contenido del archivo que se sirve.

Comprobar la ausencia de publicación de otro archivo.

Solicita `/DOCUMENTACION/nota-interna.txt`. Debe devolver 404. Es una comprobación negativa necesaria: el reto se entiende solamente si distinguimos lo que el middleware publica de lo que no publica.

También prueba una URL inexistente como `/no-existe`. Esta respuesta 404 es otra forma de ausencia, pero no demuestra nada sobre la ubicación de `nota-interna.txt`. Para esa conclusión necesitas conocer la estructura física y la configuración del middleware.

Resultado del reto documentado.

Registra tres evidencias:

Solicitud	Estado	Interpretación
`GET /Productos`	200	MVC representa el catálogo

Solicitud	Estado	Interpretación
`GET /recursos/aviso-laboratorio.txt`	200	Un archivo de `wwwroot` está disponible
`GET /DOCUMENTACION/nota-interna.txt`	404	No está publicado por el middleware actual

El reto no se resuelve simplemente porque aparece el enlace. Debes poder justificar por qué una ruta devuelve texto público y otra no, y explicar que un enlace HTML visible no establece permisos de acceso.

Comprobaciones funcionales y casos de diagnóstico

Tabla de ejecución y resultados.

Estado	Qué has activado	Verificación
Base funcional de 1.3	Ninguno	`/` y `/Productos` responden 200; se muestran tres productos
Configuración	1.3.3	El registro de inicio muestra `Curso MVC de productos`
Estilo adicional	1.3.3 + 1.3.4	`/Productos` incluye el enlace CSS adicional; el navegador carga el nuevo estilo
Guía pública	1.3.3 + 1.3.4 + 1.3.5	La navegación incluye Guía pública; el texto se descarga con 200
Reto	Todo lo anterior + RETO 1.3	Aparece Aviso de laboratorio; recurso de `wwwroot` 200 y archivo interno 404

Comprobación de continuidad: la portada y el catálogo del capítulo 1.2 deben seguir funcionando en todos los estados; no se ha desactivado ninguno de esos recorridos.

Verificación automatizada del ejercicio y el reto.

Desde la raíz del repositorio y con la aplicación manual **detenida** para dejar libre el puerto:

```
bash M01_MVC/TESTS/1.3_e2e.sh
```

La comprobación crea copias temporales, activa de manera controlada los bloques de cada etapa y comprueba con peticiones HTTP reales:

- Que las páginas de 1.1 y 1.2 continúan devolviendo HTML y muestran los productos.
- Que el registro de configuración aparece al activar el paso 1.3.3.
- Que el CSS es accesible y solo aparece enlazado en el documento cuando activamos 1.3.4.
- Que la Guía pública es accesible y se añade al menú en 1.3.5.
- Que el reto añade el enlace de aviso, cuyo archivo responde 200, mientras que el archivo exterior a `wwwroot` responde 404.

Es una prueba de integración funcional HTTP usando Kestrel; el navegador real y la comprobación visual del borde se realizan de forma guiada en clase. Una prueba HTTP del enlace y del CSS **no es equivalente a verificar su representación visual automatizada**.

Resolución de incidencias

Problemas frecuentes y soluciones.

No aparecen los tres productos al iniciar el punto 1.3. Comprueba que has abierto `PROYECTO/1.3/CursoWeb.Mvc` y no el proyecto `1.1` o el estado inicial de `1.2`. La versión 1.3 ya incorpora el catálogo funcionando.

Se obtiene una excepción al leer la configuración. Comprueba que no se alteró la sintaxis JSON de `appsettings.json`. La sección `Curso` debe encontrarse dentro de las llaves del objeto raíz y separada mediante comas correctas.

El registro no aparece en la terminal. Verifica que descomentaste la línea de `Program.cs`, que guardaste y reiniciaste el programa y que la configuración incluye `Curso:Nombre`. Comprueba también que no cambiaste el nivel de log para silenciar `Information`.

El CSS devuelve 200, pero no aparece el borde. Comprueba que activaste el `<link>` del paso 1.3.4, que realmente apunta a `/css/catalogo.css`, que recargaste la página y que no hay una regla CSS posterior que invalide el estilo. Consulta la pestaña Red y el inspector de elementos.

El enlace aparece, pero el archivo devuelve 404. Verifica la ruta relativa desde `wwwroot`: la URL no contiene `wwwroot`, sino `/recursos/guia-publica.txt` o `/recursos/aviso-laboratorio.txt`.

Al escribir la URL de `nota-interna.txt` se obtiene 404 y parece un fallo. Es el resultado esperado de la práctica. No hay un archivo dentro de `wwwroot` con esa ruta ni se ha configurado un proveedor alternativo.

Piensas que el archivo es privado porque no hay enlace. Es un error importante. Los enlaces mejoran la navegación; no definen permisos de acceso al recurso. Los archivos de `wwwroot` quedan públicamente disponibles en las condiciones demostradas.

El código de un paso afecta al siguiente. Activa los bloques en orden: configuración → CSS → Guía pública → reto. Si has dejado un comentario Razor sin cerrar, restáuralo o compara exactamente los dos delimitadores con el Markdown.

Clasificar incidencias por responsabilidad.

Fallo de compilación.

Si `dotnet build` falla, revisa primero los cambios en archivos que compilan o se procesan como parte del proyecto, como `Program.cs` y las vistas Razor. Un error de sintaxis introducido al descomentar un bloque puede impedir construir o ejecutar la aplicación.

No conviene cambiar CSS para solucionar un error de C# ni añadir nuevas rutas mientras exista un error anterior sin resolver.

Fallo de registro de configuración.

Si la aplicación compila y funciona pero no aparece el mensaje de configuración, revisa la línea `LogInformation` y la sección `Curso:Nombre`. Comprueba que el nivel de registro permita mostrar mensajes informativos.

No hay motivo para modificar `Views/Productos/Index.cshtml` en este caso: el texto esperado se escribe en el log de la aplicación y no en el HTML.

Fallo del CSS.

Si `/css/catalogo.css` no responde, revisa el archivo bajo `wwwroot/css` y `UseStaticFiles`. Si responde pero no se aplica visualmente, inspecciona la referencia `<link>` y las reglas de CSS.

El mismo síntoma visual puede exigir soluciones diferentes, por lo que el código de estado de la solicitud es una información decisiva.

Fallo del enlace público.

Si la navegación muestra Guía pública pero al pulsarla aparece 404, examina el atributo `href` y la ubicación exacta del archivo. La ruta URL debe comenzar por `/recursos/`, no por `/wwwroot/recursos/`.

Archivo de DOCUMENTACION visible inesperadamente.

En el proyecto de la práctica no se ha preparado un proveedor de archivos alternativo que publique esa carpeta. Si la URL correspondiente deja de devolver 404, comprueba que estás ejecutando la configuración original y que no se han añadido otras reglas de publicación fuera del ejercicio.

Este último caso pone de manifiesto que una prueba negativa puede descubrir una regresión de configuración, incluso cuando las páginas principales siguen respondiendo correctamente.

Resultado y continuidad

Consolidación y continuidad.

Sin modificar el código, responde y justifica sobre los archivos reales:

1. ¿Qué diferencias existen entre `CursoWeb.Mvc.csproj`, `Program.cs` y `appsettings.json`?
2. ¿Por qué `/Productos` devuelve HTML generado por MVC mientras `/css/catalogo.css` devuelve un archivo?
3. ¿Qué hace `_ViewStart.cshtml` y qué hace `_ViewImports.cshtml`?
4. ¿Por qué un archivo en `DOCUMENTACION` no queda servido como estático y por qué no deberíamos guardar secretos en un repositorio público?
5. ¿Qué pasaría si quitáramos el enlace de la Guía pública del layout: el archivo desaparecería de `wwwroot` o seguiría accesible directamente por URL?

Resultado consolidado: localizas las piezas de la aplicación y puedes explicar sus responsabilidades, su ciclo de arranque y la diferencia entre recursos físicos internos y recursos de la raíz web.

Conserva los cambios y continúa en **1.4 — Controladores, acciones, parámetros y routing**, donde ampliaremos el catálogo con una acción de detalle de producto, rutas y resultados HTTP apropiados.

Comprobar la evolución del proyecto.

Revisa nuevamente los proyectos de 1.2 y 1.3 sin mezclar sus carpetas. En el 1.2 aprendimos a pasar una lista de productos desde el controlador a una vista. En el 1.3 ese flujo continúa funcionando, pero hemos añadido recursos de configuración, un estilo y enlaces a archivos públicos.

La comparación permite distinguir una nueva funcionalidad de una nueva arquitectura. El proyecto sigue siendo MVC, no hemos añadido una base de datos ni un framework de interfaz diferente. La mayoría de los cambios se han realizado en `Program.cs`, `appsettings.json`, `Views/Shared/_Layout.cshtml` y `wwwroot`.

El alumnado debe saber explicar por qué ninguno de estos cambios obliga a reescribir `Models/Producto.cs`. La clase de dominio permanece igual porque no hemos cambiado qué representa un producto. Hemos ampliado el entorno y la presentación de la aplicación.

Una última ejecución reproducible

Desde la raíz del repositorio, con el proceso manual detenido, ejecuta el script de pruebas del capítulo:

```
bash M01_MVC/TESTS/1.3_e2e.sh
```

La prueba reconstruye cinco estados, incluido el reto, mediante copias temporales. Su finalidad es verificar que los bloques preparados se pueden activar sin romper el catálogo y que los recursos estáticos presentan los códigos de estado esperados.

No sustituye la observación con F12 ni la comprensión de qué archivo responde a cada pregunta. La batería automatizada confirma el funcionamiento técnico; el objetivo pedagógico exige relacionarlo con las responsabilidades del proyecto.

Síntesis práctica y preparación de 1.4.

Sin consultar el código durante unos minutos, explica qué archivo modificarías ante las siguientes necesidades: cambiar el texto de configuración `Curso:Nombre`, añadir un enlace común a todas las páginas, modificar la apariencia del catálogo, cambiar el nombre de un producto y publicar un nuevo archivo de texto sin crear una acción.

Después comprueba tus respuestas en el árbol del proyecto. Las ubicaciones correctas corresponden respectivamente a `appsettings.json`, `_Layout.cshtml`, CSS en `wwwroot`, código que crea los datos del producto y un recurso público bajo la raíz web, siempre que el middleware lo sirva.

Por último, explica por qué `/Productos` y `/recursos/guia-publica.txt` pueden devolver ambos 200 sin que ambos sean acciones MVC. Esta comparación será la base para comprender nuevas rutas en 1.4, donde crearemos acciones que producirán resultados HTML, texto y JSON, y aprenderemos a distinguir la selección de endpoints de la búsqueda de archivos físicos.

P1.4. Controladores, acciones, parámetros y routing

Objetivo y preparación

Objetivo principal: ampliar el catálogo MVC existente sin rehacerlo. El alumno añadirá acciones a `ProductosController`, observará varios tipos de `IActionResult`, recibirá un identificador desde la URL, construirá una ficha Razor, publicará una ruta por atributos y habilitará enlaces de detalle. El reto propone otra acción por atributos que devuelve JSON, con su código completo tanto en este documento como en el proyecto.

Teoría asociada: capítulo 1.4 del Markdown de teoría, especialmente el patrón de ruta convencional, el paso del parámetro `id`, los resultados `View`, `Content`, `BadRequest` y `NotFound`, y la comparación con `[HttpGet("/...")]`.

Tiempo orientativo de aula: 30 minutos de teoría y 65 minutos de práctica, incluido el reto breve. Los ejemplos completos y las comprobaciones adicionales permiten repasar fuera del horario lectivo sin ampliar el temario obligatorio.

El estado inicial del proyecto de **1.4** contiene una aplicación que ya muestra la lista de productos, con configuración y estilos del punto 1.3 activos. Hemos conservado los datos en memoria. Las acciones de este capítulo y el reto están **comentados**, por lo que el alumno puede habilitarlos por etapas sin añadir código desde cero.

Ubicación desde la raíz del repositorio:

```
M01_MVC/
├── PROYECTO/
│   ├── 1.1/CursoWeb.Mvc/     (estado anterior conservado)
│   ├── 1.2/CursoWeb.Mvc/     (estado anterior conservado)
│   ├── 1.3/CursoWeb.Mvc/     (estado anterior conservado)
│   └── 1.4/CursoWeb.Mvc/
│       ├── CursoWeb.Mvc.csproj
│       ├── Program.cs
│       ├── Controllers/
│       │   ├── HomeController.cs
│       │   └── ProductosController.cs
│       ├── Models/Producto.cs
│       ├── Views/
│       │   ├── Productos/Index.cshtml
│       │   ├── Productos/Detalle.cshtml
│       │   ├── Home/Index.cshtml
│       │   └── Shared/_Layout.cshtml
│       ├── appsettings.json
│       └── wwwroot/
└── TESTS/1.4_e2e.sh
```

No modificar el proyecto 1.3. El proyecto 1.4 es una continuación autocontenida del estado final funcional anterior. Las carpetas independientes permiten volver a un estado previo si se necesita reproducir la actividad de un capítulo.

Desde una terminal en la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.4/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/Productos`. Debe mostrarse la lista con **Cuaderno, Lápiz y Carpeta**. Abre también `http://localhost:5091/`: debe mantenerse la página de inicio. Este recorrido confirma que el punto de partida no ha perdido la funcionalidad desarrollada.

En este estado aún **no** está activa `/Productos/Detalle/2`: esa URL devuelve 404. Es una situación deliberada que cambiará tras activar el bloque del paso 1.4.3.

Detener el proceso con **Ctrl+C** antes de editar. Volver a iniciar la aplicación después de cada paso. No ejecutar simultáneamente dos versiones `CursoWeb.Mvc` sobre el mismo puerto.

Mapa verificable de cada paso

Cada fila permite localizar exactamente qué debe consultar o modificar el alumno. **Observar** significa que el código ya está activo y no debe descomentarse; **activar** significa que el proyecto incluye el bloque preparado. Las pruebas E2E ejecutan los estados indicados, pero no reemplazan la observación manual del navegador.

Paso	Archivo y acción del alumno	Resultado y comprobación
1.4.1	`Program.cs`; `Controllers/ProductosController.cs`. **Acción:** Examinar la ruta y la función de productos compartidos	**Resultado:** `/Productos` → 200; sin detalle activado. **Prueba:** Estado inicial de `1.4_e2e.sh`
1.4.2	`Controllers/ProductosController.cs`. **Acción:** Activar método `Estado()`	**Resultado:** `/Productos/Estado` → 200 texto plano. **Prueba:** Estado de texto en `1.4_e2e.sh`
1.4.3	`Controllers/ProductosController.cs`; `Views/Productos/Detalle.cshtml`. **Acción:** Activar `Detalle(int id)`; reutilizar la vista suministrada	**Resultado:** Detalle(2) → 200; 99 → 404; 0 → 400. **Prueba:** Estado de detalle en `1.4_e2e.sh`
1.4.4	`Controllers/ProductosController.cs`. **Acción:** Activar `Ficha(int id)` con `[HttpGet("/catalogo/ficha/{id:int}")]`	**Resultado:** Ficha(2) → 200; texto no numérico → 404. **Prueba:** Estado de ruta por atributos en `1.4_e2e.sh`
1.4.5	`Views/Productos/Index.cshtml`. **Acción:** Activar enlaces Razor de detalle dentro del bucle	**Resultado:** Enlaces a detalles 1, 2 y 3. **Prueba:** Estado de enlaces en `1.4_e2e.sh`
Reto 1.4	`Controllers/ProductosController.cs`. **Acción:** Activar `Resumen(int id)` JSON	**Resultado:** Resumen(2) → 200 JSON; 99 → 404. **Prueba:** Escenario del reto de `1.4_e2e.sh`

Regla de trazabilidad. No se considera resuelto un reto solo porque se vea su nombre en el código: se comprueba el resultado esperado con el proyecto activado. Los bloques comentados permanecen como material de ejecución del alumno; no deben sustituirse por capturas ni pseudocódigo.

Desarrollo práctico paso a paso

Paso 1.4.1 — Examinar la ruta y preparar datos compartidos

Abrir `Program.cs` y localizar esta ruta, ya activa:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Interpretación: el primer segmento identifica el controlador y, si no se proporciona, se utiliza `Home`. El segundo representa la acción y, si falta, se emplea `Index`. El tercer segmento es un valor de ruta opcional denominado `id`.

Comprueba tres URL:

```
http://localhost:5091/
http://localhost:5091/Productos
http://localhost:5091/Productos/Index
```

La primera muestra la portada; la segunda y tercera muestran el mismo listado, porque las dos llegan a la acción `Index()` de `ProductosController`.

Ahora abrir `Controllers/ProductosController.cs`. Observarás que el catálogo ya no se construye directamente en `Index`, como ocurría en 1.3. Se ha extraído a:

```
private static IReadOnlyList<Producto> ObtenerProductosEjemplo() =>
    new List<Producto>
    {
        new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
        new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
        new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
    };
```

Dentro del archivo existe además una variante histórica comentada del reto 1.2. **No la actives para este capítulo**, porque necesitamos los tres datos de partida tal como se han definido aquí.

La acción `Index` reutiliza el método:

```
[HttpGet]
public IActionResult Index()
{
    return View(ObtenerProductosEjemplo());
}
```

Explicación: el método privado reúne la definición de los datos que queremos consultar desde distintas acciones. No es una URL pública. `Index()` obtiene la colección y devuelve `View(...)`, como ya vimos en 1.2. No se ha añadido una base de datos: cada llamada a ese método construye una colección de ejemplo en memoria.

Comprobación: el contenido visible del listado no ha cambiado. Este es un ejemplo de refactorización limitada que reduce la duplicación sin alterar el resultado observado.

Profundización aplicada: Predecir el resultado antes de abrir el navegador.

El punto 1.4 introduce acciones que devuelven varios tipos de resultado. Para comprenderlas no basta con activar el código y comprobar que la página se abre. Vamos a predecir primero qué método ejecutará cada URL y qué ocurrirá con los datos recibidos.

Reconstruir la ruta convencional.

Abre `Program.cs` y localiza:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Escribe en tus apuntes qué valores toma el patrón para `/`, `/Productos` y `/Productos/Detalle/2`. En el tercer caso, el controlador es Productos, la acción es Detalle y el parámetro `id` toma el valor 2.

La ruta no ejecuta automáticamente cualquier método con un nombre parecido. Debe existir una acción adecuada en el controlador. Por eso las URL del nuevo capítulo devuelven 404 cuando sus bloques todavía están comentados.

Leer el controlador completo antes de activar acciones.

Abre `Controllers/ProductosController.cs` y localiza el método privado `ObtenerProductosEjemplo()`. Reconoce los tres datos existentes y confirma que `Index()` lo utiliza para devolver una vista de colección:

```
[HttpGet]
public IActionResult Index()
{
    return View(ObtenerProductosEjemplo());
}
```

La refactorización conserva los valores que ya aparecían en 1.3. Su finalidad es evitar copiar la colección entera en cada acción. El método privado no constituye una URL pública; ayuda a preparar los datos que las acciones necesitan.

Tabla de predicción en el estado inicial.

Antes de descomentar nada, comprueba qué ocurre con las direcciones siguientes:

```
http://localhost:5091/Productos
http://localhost:5091/Productos/Estado
http://localhost:5091/Productos/Detalle/2
http://localhost:5091/catalogo/ficha/2
```

Solo el listado está activo en este estado. Las otras tres rutas forman parte de bloques que iremos incorporando. La predicción debe anotarse **antes de realizar las solicitudes**, para que el resultado del navegador pueda confirmar o corregir tu comprensión del código.

Si observas una respuesta inesperada, verifica que has abierto `PROYECTO/1.4/CursoWeb.Mvc` y no una versión posterior del proyecto que ya tenga las acciones habilitadas.

Paso 1.4.2 — Crear una acción que devuelve texto en vez de HTML

Localizar dentro de `ProductosController.cs` el bloque:

```
/* PASO 1.4.2 INICIO
[HttpGet]
public IActionResult Estado()
{
    return Content("Catalogo MVC disponible", "text/plain");
}
PASO 1.4.2 FIN */
```

Acción: eliminar únicamente las líneas de comentario `/* PASO 1.4.2 INICIO` y `PASO 1.4.2 FIN */`. Conservar todos los caracteres del método. Se activará:

```
[HttpGet]
public IActionResult Estado()
{
```

```
    return Content("Catalogo MVC disponible", "text/plain");
}
```

Qué hace cada parte: `[HttpGet]` limita el método a solicitudes GET; `Estado()` es el nombre de la acción; `Content(...)` devuelve un cuerpo de texto con tipo `text/plain`. No creamos `Views/Productos/Estado.cshtml` porque no hemos pedido renderizar una vista.

Compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `http://localhost:5091/Productos/Estado`. Se debe mostrar **Catalogo MVC disponible**, sin menú ni layout. En las herramientas de desarrollo del navegador comprobar `GET`, código **200** y tipo de contenido `text/plain`.

Alternativa desde una segunda terminal:

```
curl -i http://localhost:5091/Productos/Estado
```

Razonamiento: `/Productos` y `/Productos/Estado` comparten controlador, pero sus acciones devuelven tipos de resultado diferentes. No hay que asumir que todos los métodos del controlador generan páginas Razor.

Detener el proceso antes de continuar.

Profundización aplicada: Diferenciar tres tipos de resultado HTTP.

Texto plano de Estado.

El bloque `PASO 1.4.2` contiene:

```
[HttpGet]
public IActionResult Estado()
{
    return Content("Catalogo MVC disponible", "text/plain");
}
```

Activa el método retirando las dos líneas de comentario que lo delimitan. Compila y ejecuta la aplicación, y solicita:

```
curl -i http://localhost:5091/Productos/Estado
```

Debes encontrar una respuesta satisfactoria y el texto `Catalogo MVC disponible`. Observa que no aparece la estructura completa del layout; no hemos utilizado `View()`.

Vista HTML de Detalle.

El bloque `PASO 1.4.3` permite activar `Detalle(int id)`. Cuando visitas `/Productos/Detalle/2` y el producto existe, la acción devuelve `View(producto)`.

La respuesta se genera utilizando `Views/Productos/Detalle.cshtml`. En ella debes localizar los elementos con identificadores `detalle-id`, `detalle-nombre` y `detalle-precio`. La página también incorpora el layout del sitio.

La diferencia respecto al método Estado es visible en el cuerpo y en el tipo de contenido de la respuesta. Ambas acciones están en el mismo controlador, pero producen formatos diferentes.

Resultado JSON del reto.

Al final del capítulo, el bloque `RETO 1.4` permite activar `Resumen(int id)`, que devuelve `Ok(new { producto.Id, producto.Nombre })`. La respuesta contiene datos JSON y no necesita una vista.

Comparación que debes poder explicar:

Acción	URL	Instrucción central	Representación
Estado	`/Productos/Estado`	`Content(..., "text/plain")`	Texto
Detalle	`/Productos/Detalle/2`	`View(producto)`	HTML
Resumen	`/catalogo/resumen/2`	`Ok(new {...})`	JSON

El objetivo no es convertir MVC en un curso completo de Web API. Solo demostramos que una acción del controlador puede elegir entre distintos resultados HTTP.

Paso 1.4.3 — Activar la ficha de detalle con parámetro id

Primero abrir `Views/Productos/Detalle.cshtml`. La vista ya existe en el proyecto, con su código completo:

```
@model CursoWeb.Mvc.Models.Producto
@{
    ViewData["Title"] = "Detalle del producto";
}
<h1 id="titulo-detalle">1.4 | Detalle de producto</h1>
<dl id="detalle-producto">
    <dt>Identificador</dt><dd id="detalle-id">@Model.Id</dd>
    <dt>Nombre</dt><dd id="detalle-nombre">@Model.Nombre</dd>
    <dt>Precio</dt><dd id="detalle-precio">@Model.Precio.ToString("0.00") €</dd>
</dl>
<p><a href="/Productos">Volver al catálogo</a></p>
```

Explicación de la vista: a diferencia de `Index.cshtml`, que recibía una lista, aquí `@model` declara un único `Producto`. Las expresiones `@Model.Id`, `@Model.Nombre` y `@Model.Precio` generan el contenido de las etiquetas HTML. El enlace final permite regresar al listado sin modificar datos.

Ahora localizar en `ProductosController.cs`:

```
/* PASO 1.4.3 INICIO
[HttpGet]
public IActionResult Detalle(int id)
{
    if (id <= 0)
    {
        return BadRequest("El identificador debe ser mayor que cero.");
    }

    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return View(producto);
}
PASO 1.4.3 FIN */
```

Acción: retirar las dos líneas que delimitan el comentario `PASO 1.4.3`, conservando el bloque interior. Se activará exactamente el método `Detalle(int id)` que se acaba de mostrar.

Explicación de funcionamiento:

1. `Detalle(int id)` recibe el identificador desde el tercer segmento de la ruta convencional.
2. `if (id <= 0)` detecta el caso que el ejercicio considera inválido y devuelve **400 Bad Request**.
3. `ObtenerProductosEjemplo().FirstOrDefault(...)` busca el producto cuyo `Id` coincida.
4. `if (producto is null) return NotFound();` genera **404** cuando no existe el elemento solicitado.
5. `return View(producto);` entrega un único modelo `Producto` a la vista correspondiente y produce HTML si todo es correcto.

Reiniciar el servidor y comprobar estas direcciones:

URL	Resultado correcto
`/Productos/Detalle/2`	200, ficha de **Lápiz** con id 2
`/Productos/Detalle/1`	200, ficha de **Cuaderno**
`/Productos/Detalle/99`	404, identificador inexistente
`/Productos/Detalle/0`	400, identificador rechazado por el controlador
`/Productos/Detalle`	400 en este ejemplo, por recibir el valor predeterminado 0

En el navegador, los códigos 400/404 pueden aparecer como páginas de error sencillas. Usar `curl -i` o F12 → Red para comprobar el **código HTTP**, que es la parte evaluada.

Qué distingue 400 de 404: en este ejemplo, 400 corresponde a un identificador que nuestro método rechaza explícitamente; 404 corresponde a un identificador que no está en la colección. Un 404 también puede producirse si ninguna ruta coincide, pero aquí, para `id=99`, el código de la acción sí se ha ejecutado.

Detener el proceso antes del siguiente paso.

Profundización aplicada: Analizar el parámetro id en varios escenarios.

Producto existente.

Con el paso 1.4.3 activado, solicita `/Productos/Detalle/2`. El valor 2 llega al parámetro `id` del método. La acción busca la instancia de producto cuyo identificador coincide y entrega ese objeto a la vista.

El archivo de vista declara:

```
@model CursoWeb.Mvc.Models.Producto
```

Esta vez no se trata de una colección. La ficha muestra los datos de un objeto individual. Comprueba que el nombre de la ficha es Lápiz y que su identificador corresponde al solicitado.

Producto inexistente.

Solicita `/Productos/Detalle/99`. El valor 99 es un entero positivo, pero no corresponde a ninguno de los tres objetos de ejemplo. La búsqueda mediante `FirstOrDefault` no encuentra un producto y la acción devuelve `NotFound()`.

Comprueba el código 404 con F12 o `curl -i`. No supongas que todo 404 significa que no existe una ruta: en este caso, la acción sí se ejecuta y decide que no hay un producto con ese identificador.

Identificador rechazado explícitamente.

Solicita `/Productos/Detalle/0`. El valor cumple el formato entero, pero la acción contiene una condición que lo rechaza:

```
if (id <= 0)
{
    return BadRequest("El identificador debe ser mayor que cero.");
}
```

El código esperado es 400. La diferencia respecto al caso anterior procede de una decisión escrita en el método, no de una propiedad automática de cualquier variable `int`.

Ausencia del segmento opcional.

Prueba `/Productos/Detalle`. En este ejemplo, con la ruta convencional y el parámetro entero no anulable, la acción puede recibir el valor predeterminado cero al faltar el segmento, y la misma regla produce 400.

Esta situación ayuda a distinguir una plantilla que permite omitir un segmento de un método que considera válido cualquier valor obtenido. La URL puede corresponder a una acción y, sin embargo, esta puede rechazar la entrada.

Construir una matriz de evidencias.

URL	Estado esperado	¿Qué línea explica el resultado?
`/Productos/Detalle/1`	200	`return View(producto)`
`/Productos/Detalle/2`	200	`return View(producto)`
`/Productos/Detalle/99`	404	`return NotFound()`
`/Productos/Detalle/0`	400	`return BadRequest(...)`

El alumnado debe localizar las líneas en el proyecto real. No basta con leer la tabla: comprueba cada resultado mediante una solicitud HTTP y relaciona el código de estado con la condición del método.

Paso 1.4.4 — Añadir una URL por atributos para reutilizar la misma vista

El listado utiliza una ruta convencional como `/Productos/Detalle/2`. Ahora incorporaremos una dirección distinta, `/catalogo/ficha/2`, usando un atributo en el método.

Localizar y activar el bloque `PASO 1.4.4`:

```
/* PASO 1.4.4 INICIO
[HttpGet("/catalogo/ficha/{id:int}")]
public IActionResult Ficha(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return View("Detalle", producto);
}
PASO 1.4.4 FIN */
```

De nuevo, eliminar únicamente las **dos líneas delimitadoras** del comentario. El resto constituye código C# ejecutable.

Aspectos clave:

- `[HttpGet("/catalogo/ficha/{id:int}")]` define la URL completa de la acción y la limita a GET. El segmento `id` tiene una restricción de entero.
- `Ficha(int id)` recibe el mismo tipo de parámetro que `Detalle(int id)`, pero desde una plantilla explícita.
- `return View("Detalle", producto);` **reutiliza** `Views/Productos/Detalle.cshtml`. Es necesario nombrarla porque, sin el nombre, MVC buscaría una vista llamada `Ficha` siguiendo la convención habitual.
- `NotFound()` mantiene el tratamiento del caso en que el identificador es entero pero no corresponde a un producto.

Reiniciar y comprobar:

```
http://localhost:5091/catalogo/ficha/2
http://localhost:5091/catalogo/ficha/99
http://localhost:5091/catalogo/ficha/no-numero
http://localhost:5091/Productos/Ficha/2
```

La primera debe mostrar la ficha de Lápiz con HTTP 200; las otras tres deben responder 404. En el segundo caso, **la acción se ejecuta pero no encuentra el producto**. En el tercero la restricción `{id:int}` impide seleccionar la acción. En el último caso no coincide la URL por atributos definida para `Ficha`: la ruta correcta no es `/Productos/Ficha/2`.

Comparación razonada: las dos acciones `Detalle(2)` y `Ficha(2)` terminan generando el **mismo HTML**, aunque fueron seleccionadas mediante URLs diferentes.

Profundización aplicada: Entender una ruta por atributos.

El bloque `PASO 1.4.4` añade una acción con esta plantilla:

```
[HttpGet("/catalogo/ficha/{id:int}")]
public IActionResult Ficha(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return View("Detalle", producto);
}
```

Retira los delimitadores del comentario y compila. Antes de abrir el navegador, explica por qué `/catalogo/ficha/2` debería alcanzar esta acción aunque el controlador se llame `ProductosController`.

La respuesta es que la plantilla del atributo define una URL explícita. No depende de construir el nombre convencional `/Productos/Ficha/2`.

Comparar dos direcciones que producen la misma vista.

Abre:

```
http://localhost:5091/Productos/Detalle/2
http://localhost:5091/catalogo/ficha/2
```

Ambas deben representar los datos del producto Lápiz utilizando `Views/Productos/Detalle.cshtml`. El método `Ficha` lo indica explícitamente mediante `View("Detalle", producto)`.

No se han creado dos archivos distintos de detalle. Esta reutilización es una buena manera de aprender que el nombre de la acción y el nombre de la vista no tienen que coincidir siempre, aunque MVC utilice esa correspondencia como convención predeterminada.

Comprobar la restricción de entero.

Solicita `/catalogo/ficha/no-numero`. La ruta especifica `{id:int}`, de modo que ese segmento no cumple la restricción. El resultado esperado es 404 por ausencia de coincidencia con el endpoint.

Compara con `/catalogo/ficha/99`. El segundo segmento es un entero válido para la ruta, pero no corresponde a un producto. La acción puede ejecutarse y devolver igualmente 404.

Pregunta: ¿por qué ambos casos generan el mismo estado aunque no tengan la misma causa? Porque 404 puede indicar que no existe una ruta compatible o que una acción seleccionada no encuentra el recurso solicitado. Para conocer el motivo debemos inspeccionar la URL, la plantilla y el método.

Distinguir una URL inexistente de una vista inexistente.

Solicita `/Productos/Ficha/2`. En esta aplicación la acción `Ficha` está asociada a una ruta por atributos absoluta, no a la dirección convencional que corresponde a su nombre. No se espera que la dirección convencional funcione.

No intentes resolverlo añadiendo una copia de `Ficha.cshtml`. La cuestión no es encontrar una vista, sino seleccionar correctamente la acción. El archivo que reutiliza la acción es `Detalle.cshtml`.

Profundización aplicada: La ruta y el nombre de la vista.

Comprobar el nombre predeterminado de la vista.

En `Detalle(int id)`, después de localizar un producto, la acción utiliza `return View(producto);`. Al no señalar un nombre, MVC aplica convenciones para buscar una vista relacionada con el nombre de la acción.

En nuestro proyecto existe `Views/Productos/Detalle.cshtml`. Debes poder explicar por qué esa ubicación corresponde al controlador Productos y a la acción Detalle.

Comprobar un nombre explícito diferente.

En `Ficha(int id)` se utiliza `return View("Detalle", producto);`. El nombre de la acción es Ficha, pero se entrega el modelo a la misma vista de Detalle. No necesitamos duplicar la plantilla HTML en un archivo `Ficha.cshtml`.

Abre las dos URLs válidas de detalle y compara su contenido. Si ambas muestran Lápiz con los mismos datos, has observado la reutilización del archivo de presentación a partir de dos acciones.

Distinguir la dirección seleccionada de la plantilla seleccionada.

La ruta `/catalogo/ficha/2` está definida por el atributo de `Ficha`. La vista seleccionada se llama Detalle. Una es una dirección externa de la aplicación y la otra es un archivo Razor utilizado para producir HTML. Sus nombres no tienen que coincidir.

Este ejercicio ayuda a evitar un error frecuente: crear nuevas vistas para resolver una ruta que no está registrada. Antes de crear archivos debemos comprobar en qué fase del recorrido se produce el problema.

Justificar por qué el retorno explicita el nombre.

Si `Ficha` devolviera `View(producto)` sin nombre, MVC podría intentar encontrar una vista asociada a la acción Ficha. El proyecto no pretende enseñar una segunda representación distinta; por eso el código usa explícitamente la cadena `"Detalle"`.

No es necesario modificar el método para comprobar este razonamiento. La práctica ya incluye la versión correcta y las pruebas automatizadas verifican que funciona con su ruta por atributos.

Paso 1.4.5 — Activar enlaces de detalle desde cada producto del listado

Una URL puede funcionar sin que exista aún un enlace visible hacia ella. Ahora añadiremos navegación al listado de forma que cada producto conduzca a su ficha.

Abrir `Views/Productos/Index.cshtml`. Dentro del `@foreach` localizar:

```
<li data-id="@producto.Id">
    @producto.Nombre — @producto.Precio.ToString("0.00") €
    @* PASO 1.4.5 (inicio)
    <a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
    PASO 1.4.5 (fin) *@
</li>
```

Acción: eliminar las dos líneas delimitadoras del comentario Razor `PASO 1.4.5`, conservando el enlace HTML:

```
<li data-id="@producto.Id">
    @producto.Nombre — @producto.Precio.ToString("0.00") €
    <a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
</li>
```

Explicación: `@producto.Id` escribe el identificador del elemento actual en la URL. El mismo bucle produce **tres enlaces**, que apuntan a `/Productos/Detalle/1`, `/Productos/Detalle/2` y `/Productos/Detalle/3`.

Reiniciar la aplicación y abrir `http://localhost:5091/Productos`. Debe aparecer el texto **Ver detalle** junto a cada producto. Pulsa el enlace de Lápiz; comprueba que la ficha muestra id 2, nombre Lápiz y precio. Utiliza el enlace **Volver al catálogo** de la vista de detalle para regresar.

Por qué activamos el enlace al final: antes de crear `Detalle(int id)`, el enlace conduciría a una dirección sin acción válida. Ahora se ha comprobado antes el endpoint y después la navegación.

Profundización aplicada: Comprobar los enlaces generados por Razor.

Una vez activados Detalle y Ficha, abre `Views/Productos/Index.cshtml` y localiza el bloque `PASO 1.4.5`:

```
@* PASO 1.4.5 (inicio)
<a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
PASO 1.4.5 (fin) *@
```

Retira las líneas delimitadoras del comentario Razor. El enlace se encuentra dentro de un bucle `foreach`, por lo que se generará uno para cada objeto.

Comprobar que los enlaces son distintos.

Abre `/Productos` e inspecciona el HTML generado. Debes encontrar los destinos:

```
/Productos/Detalle/1
/Productos/Detalle/2
/Productos/Detalle/3
```

El texto visual «Ver detalle» puede repetirse, pero el atributo `href` incorpora el valor `Id` de cada objeto. Esta diferencia permite navegar a fichas individuales sin escribir a mano tres enlaces distintos en el archivo Razor.

Navegar por dos recorridos completos.

Haz clic en el enlace del segundo producto y verifica la ficha de Lápiz. Regresa mediante `Volver al catálogo`. Después abre el enlace del primer producto y comprueba la ficha de Cuaderno.

El objetivo es observar que el controlador recibe valores distintos aunque la acción es la misma. La vista de detalle también es la misma; cambia el objeto que recibe como modelo.

Comprobar que la lista anterior no ha desaparecido.

El paso 1.4.5 añade enlaces dentro del listado existente. No debe eliminar los nombres ni el contador de tres productos. Una regresión que conserve los enlaces pero haga desaparecer los datos del listado sería un resultado incorrecto.

Reto: aplicación de los pasos

Reto 1.4 — Una acción MVC que devuelve un resultado JSON.

Enunciado y objetivo

El controlador MVC produce HTML en `Detalle` y texto plano en `Estado`. El reto consiste en publicar una tercera representación muy sencilla: un **resumen JSON** de un producto, seleccionado por su identificador, mediante una ruta por atributos.

Condiciones:

- URL `GET /catalogo/resumen/{id:int}`.
- Para un producto existente, HTTP 200 y JSON con `id` y `nombre`.
- Para un identificador entero inexistente, HTTP 404.
- Para un segmento no entero, la ruta no debe coincidir y se obtiene HTTP 404.
- No crear una vista `Resumen.cshtml` ni añadir una segunda fuente de datos.

El alumnado debe intentar explicar la solución antes de activarla. El código completo está disponible **comentado en el proyecto** y se reproduce aquí para evitar que el reto dependa de material externo.

Código del reto y explicación

Abrir `Controllers/ProductosController.cs` y localizar:

```
/* RETO 1.4 INICIO
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }
```

```
    return Ok(new { producto.Id, producto.Nombre });
}
RETO 1.4 FIN */
```

Acción 1: eliminar las líneas `/* RETO 1.4 INICIO` y `RETO 1.4 FIN */`. Se activa así **exactamente el mismo código**, sin necesidad de crear archivos adicionales.

Cómo funciona cada parte:

1. La plantilla `"/catalogo/resumen/{id:int}"` selecciona solicitudes GET cuyo último segmento puede convertirse a entero.
2. El parámetro `int id` recibe el valor extraído de la URL.
3. `ObtenerProductosEjemplo()` reutiliza el catálogo ya utilizado por `Index`, `Detalle` y `Ficha`.
4. `FirstOrDefault(p => p.Id == id)` busca el registro correspondiente.
5. Si no existe, `NotFound()` genera HTTP 404.
6. `Ok(new { producto.Id, producto.Nombre })` construye una respuesta HTTP 200 con un objeto anónimo que, con los formatos predeterminados de ASP.NET Core, se serializa en JSON.

La solución activa queda:

```
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return Ok(new { producto.Id, producto.Nombre });
}
```

Por qué no hay vista: `Ok(...)` devuelve los datos serializados; no se ha utilizado `View(...)`. La presencia de un método MVC no obliga a generar HTML. Es una comparación de resultados HTTP, no la introducción de un módulo completo de Web API.

Ejecutar, probar y razonar

Guardar, compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Desde el navegador o con `curl -i`, probar:

```
http://localhost:5091/catalogo/resumen/2
http://localhost:5091/catalogo/resumen/99
http://localhost:5091/catalogo/resumen/no-numero
```

Resultado esperado de la primera solicitud:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"id":2,"nombre":"Lápiz"}
```

El ejemplo muestra los elementos relevantes de la respuesta, no necesariamente todas las cabeceras del servidor. Las otras dos URLs deben devolver **404**, aunque por motivos distintos: ausencia de producto y ausencia de coincidencia con la restricción de ruta `id:int`.

Comprobación adicional: la URL `/Productos/Detalle/2` sigue produciendo HTML y `/Productos/Estado` sigue devolviendo texto. No hemos modificado su comportamiento.

Preguntas de consolidación y solución

¿Qué cambió respecto a `Detalle`? La selección de un resultado `Ok(objeto)` en lugar de `View(producto)`. El producto se consulta de la misma fuente de datos, pero se presenta como JSON.

¿Cuándo se ejecuta `Resumen`? Cuando llega una solicitud GET a la plantilla de ruta por atributos con un segmento `id` entero.

¿Se ejecuta la acción si escribimos `no-numero`? No, porque la ruta del reto incluye la restricción `{id:int}`. El resultado esperado es un 404 de no coincidencia de ruta.

¿Dónde está el código del reto? En este documento y en `PROYECTO/1.4/CursoWeb.Mvc/Controllers/ProductosController.cs`, delimitado por `RETO 1.4`. No se crea una solución ajena al proyecto; el alumno activa la variante incluida.

¿Cómo se prueba automáticamente? Mediante el estado `reto` de `M01_MVC/TESTS/1.4_e2e.sh`, que quita los delimitadores en una copia temporal, compila, arranca Kestrel y comprueba códigos y contenido JSON.

Ejecutar el reto JSON como consumidor.

El código del reto está preparado en `ProductosController.cs`:

```
/* RETO 1.4 INICIO
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return Ok(new { producto.Id, producto.Nombre });
}
RETO 1.4 FIN */
```

Retira las dos líneas delimitadoras. Compila, inicia y solicita:

```
curl -i http://localhost:5091/catalogo/resumen/2
```

Debe aparecer una respuesta 200 y un cuerpo JSON con las propiedades `id` y `nombre`. El valor de nombre será Lápiz. Comprueba que el tipo de contenido indica JSON.

Después prueba:

```
curl -i http://localhost:5091/catalogo/resumen/99
curl -i http://localhost:5091/catalogo/resumen/no-numero
```

Ambos casos devuelven 404, pero el primero ejecuta el método y no encuentra un producto; el segundo no cumple la plantilla de ruta con la restricción de entero.

Justificar por qué no existe Resumen.cshtml.

La acción devuelve `Ok(new {...})`, no `View(...)`. Por tanto, no necesitamos una plantilla Razor adicional para generar JSON. La respuesta es un resultado HTTP que serializa datos.

Comprobar una regresión mientras el reto está activo.

Vuelve a abrir `/Productos` y `/Productos/Detalle/2`. El listado y la ficha deben seguir funcionando, pues el reto ha añadido un endpoint sin sustituir las acciones anteriores.

La prueba automatizada del capítulo realiza esta misma idea de manera repetible: activa cada bloque, arranca Kestrel y comprueba las respuestas nuevas y las anteriores.

Comprobaciones funcionales y casos de diagnóstico

Matriz de resultados de toda P1.4.

Estado	Bloques activados	Resultado observable
Base del punto 1.4	Ninguno nuevo	`/Productos` 200 y listado; `/Productos/Detalle/2` 404
Acción de estado	1.4.2	`/Productos/Estado` 200 con `text/plain`
Detalle	1.4.2 + 1.4.3	`/Productos/Detalle/2` 200, `/99` 404 y `/0` 400
Ruta por atributos	1.4.2–1.4.4	`/catalogo/ficha/2` 200 y segmento no entero 404
Enlaces de listado	1.4.2–1.4.5	Tres enlaces «Ver detalle» hacia las fichas
Reto	Pasos anteriores + RETO 1.4	`/catalogo/resumen/2` 200 JSON y casos inexistentes 404

Una URL que retorna 404 en el estado base puede retornar 200 después de activar su acción. La diferencia es **el código habilitado**, no el navegador.

Pruebas automáticas con servidor HTTP real.

Detener la aplicación manual, volver a la **raíz del repositorio** y ejecutar:

```
bash M01_MVC/TESTS/1.4_e2e.sh
```

El script construye y arranca una aplicación en cada estado. Para evitar modificar el proyecto del alumno, crea copias temporales donde elimina únicamente los comentarios de las fases ya habilitadas. Comprueba también el listado de tres productos, la navegación, la hoja CSS y la guía pública de capítulos anteriores.

En los nuevos recorridos comprueba `GET /Productos/Estado`, `/Productos/Detalle/id`, `/catalogo/ficha/id` y `/catalogo/resumen/id`, con sus resultados 200/400/404. El reto recibe JSON real y se comprueba que contenga `id=2` y `nombre=Lápiz`.

Alcance correcto de la prueba: verifica el recorrido HTTP y el HTML resultante en un servidor Kestrel real; no reemplaza la experiencia de pulsar los enlaces en un navegador con interfaz gráfica ni constituye una prueba de seguridad de producción.

Entender lo que dice realmente un estado HTTP.

Este taller se desarrolla después de tener activos los pasos 1.4.2–1.4.5. No añade nuevos controladores ni vistas: utiliza los métodos ya incluidos en el proyecto para contrastar hipótesis. Su finalidad es que el

alumno deje de utilizar el código HTTP 200 como única prueba de que una funcionalidad está bien realizada.

Comprobar tres dimensiones de la misma respuesta.

Con la aplicación en ejecución, utiliza:

```
curl -i http://localhost:5091/Productos/Detalle/2
```

De la salida identifica tres informaciones. La primera es el **código de estado**, que debería indicar éxito. La segunda es el **tipo de contenido**, que debe corresponder a HTML. La tercera es el **contenido de la ficha**, donde deben aparecer el identificador 2 y el nombre Lápiz.

Si la respuesta fuera 200 pero mostrara Cuaderno, no sería correcta. El endpoint estaría respondiendo, pero habría una discrepancia entre el parámetro solicitado y el objeto representado. Si se devolviera texto plano en vez de HTML, tampoco cumpliría el resultado esperado del ejercicio.

Anota la correspondencia entre la URL solicitada, el valor recibido como `id` y la propiedad `Id` del objeto. Es posible escribir una acción que devuelva siempre el primer producto; por eso debemos comprobar el dato y no solamente la existencia de la página.

Diferenciar una respuesta de error diseñada de un fallo de ejecución.

Solicita:

```
curl -i http://localhost:5091/Productos/Detalle/99
curl -i http://localhost:5091/Productos/Detalle/0
```

La primera debería devolver 404. La acción ha sido seleccionada correctamente, pero no encontró un producto con id 99. La segunda debería devolver 400 debido a la regla del propio método que rechaza valores menores o iguales que cero.

En ninguna de las dos situaciones resulta correcto afirmar automáticamente que «MVC se ha caído». El servidor continúa atendiendo solicitudes y ha producido respuestas HTTP previstas por el diseño de la práctica.

Comprueba además `/Productos/Detalle/2` después de solicitar los dos casos de error. La ficha válida debe seguir funcionando; los resultados negativos no deberían alterar el catálogo fijo en memoria.

Comprobar que un error de ruta no es un error de dato.

Una vez activada `Ficha`, solicita:

```
curl -i http://localhost:5091/catalogo/ficha/no-numero
curl -i http://localhost:5091/catalogo/ficha/99
```

Las dos solicitudes deben devolver 404, pero por caminos diferentes. La primera no satisface la restricción `{id:int}`; la segunda contiene un número entero y puede seleccionar la acción, pero el objeto no existe.

La diferencia es invisible si únicamente miramos una página de error del navegador. Para explicarla necesitamos leer el atributo de ruta y el código de búsqueda del controlador.

Redactar una incidencia reproducible.

Escribe una breve descripción de un caso 404 siguiendo este formato: dirección solicitada, bloque de código activado, acción que se esperaba seleccionar, resultado realmente observado y por qué es correcto o incorrecto según el enunciado.

No incluyas afirmaciones como «hay un problema de rutas» sin especificar la URL. El objetivo es practicar una comunicación técnica que permita a otro alumno reproducir la situación.

Seguir un producto a través de distintas representaciones.

Este taller se ejecuta después de activar también el reto `Resumen`. El proyecto incluye entonces varias maneras de consultar el mismo producto sin cambiar el origen de los datos.

Preparar la tabla de correspondencias.

Elige Lápiz, cuyo identificador es 2. Comprueba que aparece en la función privada que prepara el catálogo y que las acciones reutilizan la misma colección.

Después consulta:

```
http://localhost:5091/Productos
http://localhost:5091/Productos/Detalle/2
http://localhost:5091/catalogo/ficha/2
http://localhost:5091/catalogo/resumen/2
```

La primera dirección muestra una colección que contiene Lápiz. La segunda y tercera representan una ficha HTML del mismo producto, aunque se seleccionan mediante rutas diferentes. La última devuelve un resumen JSON.

Relacionar el formato con el método de retorno.

Abre `ProductosController.cs` y localiza:

```
return View(ObtenerProductosEjemplo());
```

```
return View(producto);
```

```
return View("Detalle", producto);
```

```
return Ok(new { producto.Id, producto.Nombre });
```

Estos fragmentos corresponden a resultados distintos y ya forman parte de las acciones preparadas en el proyecto. No son cuatro formas equivalentes de escribir la misma instrucción. El primero entrega una colección a la vista de listado; el segundo entrega un producto a la vista de detalle por convención; el tercero selecciona explícitamente esa vista; el cuarto devuelve una representación de datos.

En tus apuntes anota el nombre de la acción correspondiente a cada retorno y la URL desde la que puedes observar el resultado.

Comparar el modelo enviado con el resultado visible.

La vista de detalle recibe un `Producto` con sus propiedades Id, Nombre y Precio. El resumen JSON del reto contiene únicamente Id y Nombre, porque el objeto anónimo de `Ok(new {...})` selecciona esas propiedades.

Esto no significa que el producto haya perdido su precio. Significa que una representación concreta ha elegido publicar un subconjunto de los datos. La selección de formato y de propiedades forma parte del contrato de respuesta del endpoint.

Explicar cómo se reutiliza la información.

Todas las acciones consultan la misma función `ObtenerProductosEjemplo()`. Al construir una sola colección de referencia se reduce el riesgo de que el listado utilice unos datos y la ficha otros. El proyecto no tiene persistencia: cada solicitud reconstruye los objetos fijos.

No confundas compartir una función que crea datos de ejemplo con compartir un repositorio de base de datos. En M02 cambiará la forma de obtener los productos, pero será útil mantener claras las responsabilidades de la acción, la vista y el mecanismo de almacenamiento.

Casos que no deben producirse.

La finalidad de una revisión técnica no es demostrar únicamente que las rutas felices funcionan. También debemos pensar en comportamientos inesperados que el ejercicio no permite.

No debe aparecer una ficha para un id inexistente.

Si `/Productos/Detalle/99` mostrara una ficha con datos del producto 1, habría una discrepancia. La condición del controlador establece que, cuando no se encuentra el objeto, se devuelve `NotFound()`. El test del capítulo comprueba ese comportamiento.

No debe aparecer HTML completo en la acción de Estado.

La respuesta de `/Productos/Estado` utiliza `Content(..., "text/plain")`. Si mostrara el menú, las hojas de estilo y el cuerpo de una vista Razor, no estaríamos observando el tipo de resultado previsto por el ejercicio.

Esta situación sería diferente de un texto visual que parece simple: debemos comprobar el tipo de contenido y no inferirlo solo por la apariencia del navegador.

No debe ser accesible Ficha mediante una ruta no definida.

En este proyecto, `Ficha` tiene una plantilla absoluta `/catalogo/ficha/{id:int}`. La dirección `/Productos/Ficha/2` no forma parte del recorrido previsto. Si devolviera una ficha, habría que revisar si se ha añadido otra ruta o cambiado el método más allá de lo indicado.

No se trata de prohibir esa dirección en cualquier aplicación MVC. Se trata de comprobar la configuración concreta que el alumno está estudiando.

No debe confundirse la respuesta JSON con una vista Razor.

El reto utiliza `Ok(new { producto.Id, producto.Nombre })`. Crear un archivo `Resumen.cshtml` no es necesario y no representa la solución del enunciado.

La comprobación debe confirmar una respuesta JSON válida con las propiedades correctas, no simplemente la presencia del nombre del producto en cualquier cuerpo de respuesta.

No deben desaparecer funcionalidades de capítulos anteriores.

El listado `/Productos`, la portada `/` y los recursos CSS deben continuar funcionando después de activar las acciones nuevas. Si alguno deja de funcionar, hay una regresión que el alumno debe investigar antes de dar el capítulo por terminado.

Esta es la razón por la que el script `1.4_e2e.sh` prueba también funcionalidades que no se han introducido en 1.4. Una ampliación correcta conserva los contratos previos.

Resolución de incidencias

Problemas frecuentes y soluciones.

La aplicación no compila al activar una acción. Revisa que has quitado los dos delimitadores `/* ... */` del bloque y que no has añadido o borrado llaves accidentalmente. Compila después de cada paso para acotar el error.

`/Productos/Estado` sigue devolviendo 404. Verifica que la acción `Estado` está activa y que estás ejecutando `PROYECTO/1.4`, no la copia anterior de 1.3. Un proceso previo podría estar usando el puerto 5091.

`/Productos/Detalle/2` devuelve 404 aunque el id existe. Comprueba que activaste la acción `Detalle`, que la URL usa la ruta convencional correcta y que la lista contiene el id 2.

Un detalle con id 0 devuelve 400. Es el comportamiento correcto del ejercicio: `Detalle(int id)` tiene una comprobación que rechaza valores menores o iguales que cero. No debemos cambiarla para forzar una respuesta 200.

`/catalogo/ficha/2` devuelve 404. La acción `Ficha` debe estar activa y llevar el atributo `[HttpGet("/catalogo/ficha/{id:int}")]`. No basta con agregar una vista llamada Ficha; aquí reutilizamos explícitamente la vista Detalle.

`/Productos/Ficha/2` no funciona. La acción `Ficha` ha definido una URL absoluta por atributos, no la URL convencional que corresponde a su nombre. La petición correcta es `/catalogo/ficha/2`.

Los enlaces de detalle no aparecen. Revisar que se han quitado ambos delimitadores del comentario Razor `PASO 1.4.5` dentro del `@foreach` de `Views/Productos/Index.cshtml`. Una acción que funciona no genera por sí misma enlaces en el listado.

El reto devuelve HTML en vez de JSON. Comprueba que utilizas `Ok(new { ... })` y no `View(producto)`. Inspecciona la cabecera `Content-Type`; en el ejercicio esperamos un formato JSON.

En la respuesta JSON los nombres aparecen en minúscula inicial. Es la convención de serialización habitual, no un cambio del modelo C#. La propiedad `Id` de C# puede aparecer como `id` en JSON.

Resolver incidencias con un criterio claro.

Si `/Productos/Estado` devuelve 404, confirma que activaste la acción Estado y que la URL utiliza el controlador correcto. No es necesario crear una vista.

Si `/Productos/Detalle/2` falla después de activar el método, comprueba la firma `Detalle(int id)`, el valor de la ruta y la existencia de `Views/Productos/Detalle.cshtml`. Distingue si el fallo es de selección de acción, de búsqueda de producto o de vista.

Si `/catalogo/ficha/2` devuelve 404, comprueba el atributo de ruta absoluta. No añadas de forma apresurada otra regla convencional para hacer funcionar una URL equivocada.

Si el reto devuelve HTML en lugar de JSON, revisa `Ok(new { ... })` y confirma que no has copiado `View(producto)` desde la acción Detalle.

Si el listado no muestra los enlaces, revisa el comentario Razor `PASO 1.4.5` y confirma que la etiqueta está dentro del `foreach`.

Si una acción devuelve 200 pero el contenido es incorrecto, no consideres la prueba superada solamente por el código de estado. Comprueba qué producto se representó y qué formato devolvió el servidor.

Cada incidencia permite asociar un síntoma con una responsabilidad. Esta habilidad resulta más valiosa que memorizar de forma aislada los nombres de todos los métodos de `Controller`.

Resultado y continuidad

Cierre y continuidad.

Al terminar, el alumno debe poder explicar **qué acción MVC se ejecuta para cada URL, cómo obtiene el id y por qué devuelve un resultado HTTP concreto**. El listado sigue funcionando y permite navegar a fichas individuales. Se han utilizado el mismo modelo y la misma vista para distintos recorridos sin una base de datos.

El reto demuestra, mediante código real comentado y comprobable, que un controlador MVC también puede producir JSON. Todos los fragmentos que se han pedido ejecutar existen en el proyecto del capítulo.

Conserva este estado funcional. En el punto **1.5** profundizaremos en las vistas Razor, modelos fuertemente tipados, layouts y vistas parciales, utilizando las fichas que acabamos de construir. Aún no añadiremos formularios ni persistencia: son actividades de M02.

Explicar el recorrido de un producto.

Elige Lápiz y describe las tres rutas mediante las que podemos obtener información de ese producto después de completar el capítulo:

- `/Productos/Detalle/2`: HTML de una ficha seleccionada por ruta convencional.
- `/catalogo/ficha/2`: el mismo HTML obtenido mediante una ruta por atributos.
- `/catalogo/resumen/2`: JSON con un resumen del producto, tras activar el reto.

Para cada una, identifica el método que se ejecuta, la instrucción que devuelve el resultado y el formato esperado. Después justifica por qué dos direcciones pueden usar una misma vista y por qué la tercera no necesita ninguna.

Ejecuta por último, desde la raíz del repositorio y con el servidor manual detenido:

```
bash M01_MVC/TESTS/1.4_e2e.sh
```

La secuencia automatizada comprueba seis estados, incluido el reto. No sustituye la explicación personal: un resultado correcto debe acompañarse de la capacidad de predecir qué hará cada endpoint antes de solicitarlo.

Una vez comprendidas las acciones y rutas, el siguiente capítulo se centrará en la presentación Razor. Conservaremos los mismos endpoints de catálogo y detalle y aprenderemos a representar productos mediante layouts, secciones y vistas parciales reutilizables.

Cierre ampliado del capítulo 1.4.

Describe por escrito el recorrido de `GET /catalogo/ficha/2` desde la URL hasta el HTML final. Debes señalar la plantilla del atributo, el parámetro `id`, el método privado que obtiene productos, la búsqueda de Lápiz y la instrucción `View("Detalle", producto)`.

Después describe `GET /catalogo/resumen/2` y explica qué cambia: la acción se selecciona mediante otra plantilla, pero utiliza el mismo origen de datos y elige una respuesta JSON en vez de una vista.

Finalmente, compara `/Productos/Detalle/0`, `/Productos/Detalle/99` y `/catalogo/ficha/no-numero`. Cada uno exige una explicación diferente aunque todos sean casos que no muestran una ficha válida.

Para una comprobación técnica adicional, ejecuta el E2E del capítulo desde la raíz del repositorio:

```
bash M01_MVC/TESTS/1.4_e2e.sh
```

Al cerrar el taller, el alumno debería saber predecir qué acción recibe cada URL, qué dato utiliza y por qué produce un estado y formato de respuesta determinados. Con esa base será posible profundizar en Razor y las vistas parciales del siguiente capítulo sin confundir enrutamiento con representación.

P1.5. Vistas Razor tipadas, layouts y vistas parciales

Objetivo y preparación

Objetivo: reconocer los elementos de la sintaxis Razor que usamos para construir HTML, distinguir el modelo de una vista de lista del modelo de una ficha, ampliar un layout mediante una sección opcional y reutilizar una vista parcial tipada para mostrar productos en el listado y en la ficha individual.

Reto con código: incorporar en la parcial una etiqueta visual que aparezca cuando el precio sea inferior a tres unidades. El reto está preparado como bloque Razor comentado dentro del proyecto y debe verificarse tanto en el listado como en el detalle. Aquí se proporciona enunciado, solución, explicación y comandos exactos.

Teoría relacionada: capítulo 1.5 del Markdown teórico de M01; conviene tener abiertos los apartados sobre `@model`, expresiones, `@if`, `@foreach`, `@section`, `@RenderBody()` y el Partial Tag Helper.

Tiempo orientativo de aula: 20 minutos de teoría y 50 de práctica incluyendo reto. Las explicaciones desarrolladas permiten estudio individual; las comprobaciones automáticas no consumen tiempo de aula.

Estado inicial: el proyecto del capítulo **1.5** contiene las páginas completas de P1.4: listado con tres enlaces de detalle, acción `Detalle(int id)`, acción `Ficha(int id)` por atributos y la acción de texto `Estado()`. El reto opcional de P1.4 sigue comentado. Lo nuevo de P1.5 está preparado en archivos Razor con comentarios que el alumno irá retirando.

El proyecto se encuentra en:

```
M01_MVC/PROYECTO/1.5/CursoWeb.Mvc/
├── Controllers/
│   ├── HomeController.cs
│   └── ProductosController.cs
├── Models/
│   └── Producto.cs
├── Views/
│   ├── _ViewStart.cshtml
│   ├── _ViewImports.cshtml
│   ├── Productos/
│   │   ├── Index.cshtml
│   │   └── Detalle.cshtml
│   └── Shared/
│       ├── _Layout.cshtml
│       └── _TarjetaProducto.cshtml
├── wwwroot/
│   ├── css/
│   │   ├── site.css
│   │   └── catalogo.css
│   └── recursos/
└── CursoWeb.Mvc.csproj
```

No editar el proyecto 1.4 para esta práctica: el de 1.5 conserva la evolución anterior y permite activar los siguientes pasos sin reconstruir otras versiones. El archivo parcial `_TarjetaProducto.cshtml` y sus estilos **ya existen**, aunque inicialmente las vistas todavía no lo incluyen.

Abrir `CursoWeb.Mvc.csproj` en Visual Studio, o ejecutar desde la raíz del repositorio:

```
cd M01_MVC/PROYECTO/1.5/CursoWeb.Mvc
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

En el navegador comprobar:

```
http://localhost:5091/
http://localhost:5091/Productos
http://localhost:5091/Productos/Detalle/2
http://localhost:5091/catalogo/ficha/2
```

La portada, el catálogo y las dos rutas de detalle deben funcionar. `/Productos` muestra Cuaderno, Lápiz y Carpeta con enlaces «Ver detalle», pero todavía **no** muestra las tarjetas adicionales ni el mensaje de catálogo completo.

Detener la ejecución antes de modificar los bloques. En los pasos siguientes puedes conservar la misma terminal y lanzar nuevamente `dotnet run --no-launch-profile --urls http://localhost:5091` después de compilar.

Mapa verificable de cada paso

Cada fila permite localizar exactamente qué debe consultar o modificar el alumno. **Observar** significa que el código ya está activo y no debe descomentarse; **activar** significa que el proyecto incluye el bloque preparado. Las pruebas E2E ejecutan los estados indicados, pero no reemplazan la observación manual del navegador.

Paso	Archivo y acción del alumno	Resultado y comprobación
1.5.1	`Views/Productos/Index.cshtml` ; `Views/Productos/Detalle.cshtml` . **Acción:** Distinguir modelo colección de un modelo individual	**Resultado:** Catálogo y detalle presentan los tipos adecuados. **Prueba:** Estado inicial de `1.5_e2e.sh`
1.5.2	`Views/Productos/Index.cshtml` . **Acción:** Activar condición `Model.Count >= 3`	**Resultado:** Mensaje `catalogo-completo` en el listado. **Prueba:** Estado condicional de `1.5_e2e.sh`
1.5.3	`Views/Shared/_Layout.cshtml` ; `Views/Productos/Detalle.cshtml` . **Acción:** Activar definición y representación de `NotasPagina` conjuntamente	**Resultado:** Nota en detalle, otras páginas continúan funcionando. **Prueba:** Estado de secciones de `1.5_e2e.sh`
1.5.4	`Views/Productos/Index.cshtml` ; `Views/Shared/_TarjetaProducto.cshtml` . **Acción:** Activar Partial Tag Helper dentro de `foreach`	**Resultado:** Tres tarjetas con `data-producto-id` distinto. **Prueba:** Estado de tarjetas de `1.5_e2e.sh`
1.5.5	`Views/Productos/Detalle.cshtml` ; `Views/Shared/_TarjetaProducto.cshtml` . **Acción:** Activar reutilización de parcial en ficha	**Resultado:** Una tarjeta del producto solicitado. **Prueba:** Estado de reutilización de `1.5_e2e.sh`
Reto 1.5	`Views/Shared/_TarjetaProducto.cshtml` . **Acción:** Activar comparación `Model.Precio < 3m`	**Resultado:** Etiqueta solo en Lápiz; ausente en Cuaderno y Carpeta. **Prueba:** Escenario del reto de `1.5_e2e.sh`

Regla de trazabilidad. No se considera resuelto un reto solo porque se vea su nombre en el código: se comprueba el resultado esperado con el proyecto activado. Los bloques comentados permanecen como material de ejecución del alumno; no deben sustituirse por capturas ni pseudocódigo.

Desarrollo práctico paso a paso

Paso 1.5.1 — Identificar el tipo de modelo y el origen del título de página

Abrir `Views/Productos/Index.cshtml`. Al principio del archivo observarás:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
@{
    ViewData["Title"] = "Catálogo de productos";
}
```

La directiva `@model` informa al motor Razor de que recibimos **una lista** de `Producto`, o un valor nulo si no se ha proporcionado. El bloque `@{ ... }` ejecuta C# para establecer un título; no dibuja por sí mismo una etiqueta HTML.

Más abajo, la vista tiene una condición `@if (Model is null)`. Cuando no hay modelo presenta un mensaje, y cuando sí lo hay escribe `@Model.Count` y recorre la lista con `@foreach (var producto in Model)`.

Ahora abrir `Views/Productos/Detalle.cshtml`. Su directiva es diferente:

```
@model CursoWeb.Mvc.Models.Producto
@{
    ViewData["Title"] = "Detalle del producto";
}
```

La ficha recibe **un solo producto**. Por eso consulta `@Model.Id`, `@Model.Nombre` y `@Model.Precio` sin recorrer una lista.

Finalmente abre `Views/Shared/_Layout.cshtml` y localiza **por separado** estas dos instrucciones ya existentes: `<title>@ViewData["Title"] - Curso Web .NET 8</title>` y `@RenderBody()`. No son líneas consecutivas ni forman un bloque nuevo que debas copiar.

La etiqueta `<title>` de cada respuesta combina el valor de `ViewData` de la vista con el texto común. `@RenderBody()` establece dónde se inserta el cuerpo de la vista dentro del documento principal.

Comprobación sin editar nada: abre `/Productos` y `/Productos/Detalle/2` y compara los títulos de las pestañas. Después utiliza F12 para inspeccionar el HTML y localizar las etiquetas `<h1>`, `<title>` y los nombres de los productos. En la respuesta HTML no se debe encontrar código ejecutable `@Model.Nombre`: Razor ya lo ha evaluado.

Resultado del paso: identificas dos contratos de modelo diferentes y una responsabilidad del layout. No se requiere descomentar bloques todavía.

Profundización aplicada: Seguir el HTML desde la vista hasta el navegador.

El último punto de M01 utiliza todas las piezas estudiadas: rutas, controladores, modelos, vistas y layout. Este laboratorio profundiza en la composición de una respuesta HTML sin introducir formularios ni JavaScript de aplicación.

Identificar dónde se produce cada parte del documento.

Abre el proyecto `PROYECTO/1.5/CursoWeb.Mvc` y localiza estas tres vistas:

```
Views/Productos/Index.cshtml
Views/Productos/Detalle.cshtml
Views/Shared/_Layout.cshtml
```

Abre `Index.cshtml` y señala la directiva `@model`, el valor de `ViewData["Title"]`, la condición de modelo nulo y el `foreach` que genera los productos. Después abre `_Layout.cshtml` y busca `@RenderBody()`.

Explica el recorrido: la acción `Index` entrega una lista de productos, la vista representa su contenido y el layout coloca ese contenido dentro de una página con navegación, estilos y pie compartidos.

No es necesario modificar ningún archivo para responder. La finalidad es reconocer qué parte de la respuesta se define en la vista y cuál es responsabilidad del layout.

Comprobar el resultado con F12.

Ejecuta la aplicación:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abre `http://localhost:5091/Productos` y accede a F12 → Red. Selecciona la solicitud del documento y examina el HTML recibido. Debe contener el listado y las etiquetas del layout.

Busca `@Model.Count` como texto literal. No debería aparecer, porque Razor ya ha evaluado la expresión en el servidor. En su lugar encontrarás el número concreto de productos representado dentro del párrafo correspondiente.

Comparar dos contratos de vista.

La vista del catálogo declara:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

La de detalle declara:

```
@model CursoWeb.Mvc.Models.Producto
```

Abre `/Productos/Detalle/2` y comprueba que se representa un único producto. En el código de la vista, localiza `@Model.Id` y `@Model.Nombre`. No existe allí un bucle que recorra una lista de productos, porque el contrato es diferente.

Pregunta de cierre: ¿qué ocurriría si intentáramos ejecutar en la vista de detalle el mismo `foreach` diseñado para la colección? El tipo de modelo no sería el esperado por ese fragmento. La sintaxis y el contrato deben corresponderse con los datos que el controlador entrega.

Paso 1.5.2 — Incorporar una condición Razor de presentación

Abrir `Views/Productos/Index.cshtml` y localizar dentro del bloque `else` el fragmento comentado:

```
@* PASO 1.5.2 (inicio)
@if (Model.Count >= 3)
{
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
PASO 1.5.2 (fin) *@
```

Acción: eliminar únicamente las dos líneas delimitadoras del comentario Razor que contienen `PASO 1.5.2`. Mantener la instrucción `@if` y el elemento HTML. El código activo queda:

```
@if (Model.Count >= 3)
{
```

```
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
```

Explicación: `Model.Count` obtiene la cantidad de productos que entregó el controlador. La condición compara la cantidad con 3 y decide si incluye el párrafo en el HTML final. No modifica la lista ni crea productos nuevos. El bloque está dentro del `else` que garantiza que `Model` no es nulo.

Compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `/Productos`. Debe aparecer el mensaje **El catálogo contiene al menos tres productos**, junto al contador de tres productos y la lista existente. La ficha `/Productos/Detalle/2` continúa igual, porque su vista es distinta.

Comprobación: en F12 → Red/Network → respuesta de `/Productos`, buscar `id="catalogo-completo"`. Este elemento no estaba presente en el estado inicial; aparece exclusivamente después de activar la condición.

Detener la ejecución para pasar al siguiente ejercicio.

Profundización aplicada: Condiciones Razor sin cambiar el modelo.

Activar un mensaje según la cantidad.

En `Views/Productos/Index.cshtml` localiza el bloque `PASO 1.5.2`:

```
@* PASO 1.5.2 (inicio)
@if (Model.Count >= 3)
{
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
PASO 1.5.2 (fin) *@
```

Retira los delimitadores del comentario Razor y conserva la condición. Compila y ejecuta. En `/Productos` debe aparecer el mensaje porque el catálogo tiene tres elementos.

No cambies la lista en `ProductosController`: el objetivo es demostrar que la vista puede decidir si representa una pieza de marcado según los datos recibidos.

Inspeccionar el fragmento generado.

En la respuesta HTML busca `id="catalogo-completo"`. Verifica que el párrafo aparece después de activar el bloque. La expresión `@if` no se envía como código ejecutable al navegador.

Compara esta comprobación con la realizada en el punto 1.2, donde primero aparecía un contador y después activábamos el bucle de productos. En ambos casos, Razor utiliza datos del modelo para determinar contenido de la respuesta.

Interpretar correctamente la comprobación de nulos.

El `@if (Model.Count >= 3)` se ha incluido dentro de un `else` que corresponde a `if (Model is null)`. Eso es importante: evita acceder a `Count` cuando el modelo es nulo.

No cambies la organización de las llaves para ahorrar líneas. La estructura existente es intencionada y ayuda a razonar sobre los dos estados de modelo que vimos anteriormente.

Paso 1.5.3 — Extender el layout mediante una sección Razor opcional

Este paso requiere **dos cambios coordinados**. En uno, el layout establece dónde mostrará notas adicionales; en el otro, la vista de detalle define qué texto debe ocupar esa zona.

Primer archivo: `Views/Shared/_Layout.cshtml`. Localizar:

```
@* PASO 1.5.3 (inicio)
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
PASO 1.5.3 (fin) *@
```

Quitar solo las líneas delimitadoras del comentario. El bloque activo queda:

```
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
```

Qué hace: `RenderSectionAsync("NotasPagina", required: false)` representa una sección de nombre `NotasPagina` si la vista que se está renderizando la ha definido. Como la sección es **opcional**, no se exige a todas las páginas del sitio que la declaren.

Segundo archivo: `Views/Productos/Detalle.cshtml`. Localizar el bloque:

```
@* PASO 1.5.3 (inicio)
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
PASO 1.5.3 (fin) *@
```

Retirar sus dos delimitadores y conservar:

```
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
```

Qué hace: la página de detalle define el contenido que el layout insertará en su zona de notas. Este bloque no genera automáticamente un segundo documento HTML; proporciona un fragmento que el layout representa en la posición elegida.

Importante: en esta práctica, **activa ambos bloques antes de compilar**. Una sección definida por una vista debe ser tratada por su layout; no queremos dejar una sección definida pero sin destino.

Compilar, iniciar y visitar `/Productos/Detalle/2`. Debe seguir mostrando la ficha de Lápiz y, fuera de su cuerpo principal, el mensaje **Detalle construido con una vista Razor tipada**.

A continuación visita `/Productos` y `/`. Ambas deben funcionar sin definir la sección, porque el layout utiliza `required: false`. Es normal que la estructura externa del layout exista aunque una página no aporte contenido a dicha zona.

Resultado de aprendizaje: has observado la diferencia entre una vista concreta y un diseño compartido, y has visto cómo una sección opcional permite personalizar una región de ese diseño.

Detener el servidor antes de continuar.

Profundización aplicada: Comprender una sección de layout.

Localizar el espacio de representación.

Abre `Views/Shared/_Layout.cshtml` y busca el comentario `PASO 1.5.3`:

```
@* PASO 1.5.3 (inicio)
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
PASO 1.5.3 (fin) *@
```

La instrucción `RenderSectionAsync` declara un lugar para mostrar contenido que algunas vistas pueden definir. El nombre `NotasPagina` debe coincidir con el utilizado por la vista.

Localizar el contenido proporcionado por la ficha.

En `Views/Productos/Detalle.cshtml` busca:

```
@* PASO 1.5.3 (inicio)
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
PASO 1.5.3 (fin) *@
```

Este bloque proporciona el texto que el layout representará. El archivo de detalle sigue conteniendo también sus elementos principales con `Id`, `Nombre` y `Precio`.

Activar los dos bloques coordinadamente.

Retira los delimitadores del comentario en ambos archivos antes de compilar. No actives únicamente la definición de la sección y dejes el layout sin el marcador que la consume.

Compila e inicia. Abre `/Productos/Detalle/2`. Debes encontrar el mensaje de la sección dentro de la estructura compartida, separado del cuerpo principal de la ficha.

Después abre `/Productos` y `/`. Las páginas deben seguir funcionando aunque no definan `NotasPagina`, porque el layout usa `required: false`.

Demostrar que no se ha creado una vista nueva.

La sección no requiere una URL independiente ni un nuevo controlador. El contenido se define en `Detalle.cshtml` y se inserta donde el layout lo solicita.

Esta observación diferencia las secciones de la navegación. No hemos creado una página `/NotasPagina` ni un archivo `NotasPagina.cshtml`. Hemos ampliado la composición de una respuesta HTML existente.

Paso 1.5.4 — Representar una vista parcial por cada producto del catálogo

El proyecto incorpora una **vista parcial tipada** en `Views/Shared/_TarjetaProducto.cshtml`. Abre el archivo y localiza:

```
@model CursoWeb.Mvc.Models.Producto

<article class="tarjeta-producto" data-producto-id="@Model.Id">
    <h3>@Model.Nombre</h3>
    <p class="tarjeta-precio">Precio: @Model.Precio.ToString("0.00") €</p>
    <a href="/Productos/Detalle/@Model.Id">Ver detalle de @Model.Nombre</a>
</article>
```

En el archivo real hay comentarios docentes y un bloque del reto comentado dentro del artículo. **No actives todavía el reto**. La parcial ya contiene todo lo necesario para mostrar una tarjeta de un producto.

Qué significa el código: `@model Producto` exige un objeto `Producto` individual; `@Model.Nombre` y `@Model.Precio` generan texto; el atributo `data-producto-id` permite observar qué producto se ha pasado. El enlace conduce a su detalle. No hay lógica de acceso a datos ni acciones HTTP en la parcial.

Abre ahora `Views/Productos/Index.cshtml` y localiza:

```
@* PASO 1.5.4 (inicio)
<section id="tarjetas-catalogo" aria-label="Tarjetas reutilizables de productos">
    <h2>Vista de tarjetas</h2>
    @foreach (var producto in Model)
    {
        <partial name="_TarjetaProducto" model="producto" />
    }
</section>
PASO 1.5.4 (fin) *@
```

Acción: eliminar las dos líneas delimitadoras del comentario `PASO 1.5.4`, manteniendo las demás intactas.

Dentro del `foreach`, la variable `producto` se refiere a un objeto de la colección. El Partial Tag Helper `<partial name="_TarjetaProducto" model="producto" />` entrega ese objeto a `_TarjetaProducto.cshtml`, que lo recibe como su `Model`.

¿Por qué funciona? Porque en `Views/_ViewImports.cshtml` ya existe el registro de Tag Helpers de MVC:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Sin esa directiva, la sintaxis del elemento `<partial>` no se procesaría como esperamos. No hace falta instalar otra biblioteca para el curso.

Compilar y ejecutar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Abrir `/Productos`. Debajo de la lista clásica debe aparecer la sección **Vista de tarjetas**, con **tres tarjetas**: Cuaderno, Lápiz y Carpeta. Cada tarjeta muestra nombre, precio y un enlace de detalle. El listado anterior continúa visible para comparar las dos formas de representación.

Comprobación técnica: en F12 → Red → Response, localizar `id="tarjetas-catalogo"` y exactamente **tres** elementos con `class="tarjeta-producto"`. No se han generado tres peticiones HTTP nuevas para las tarjetas: son fragmentos de una única respuesta HTML.

Profundización aplicada: Utilizar una vista parcial en una colección.

Leer la parcial antes de invocarla.

Abre `Views/Shared/_TarjetaProducto.cshtml`. Identifica:

```
@model CursoWeb.Mvc.Models.Producto

<article class="tarjeta-producto" data-producto-id="@Model.Id">
    <h3>@Model.Nombre</h3>
    <p class="tarjeta-precio">Precio: @Model.Precio.ToString("0.00") €</p>
```

```
    <a href="/Productos/Detalle/@Model.Id">Ver detalle de @Model.Nombre</a>
</article>
```

El archivo real contiene también un bloque comentado del reto. No lo actives todavía. Comprueba que la parcial recibe un **objeto Producto**, no la colección completa que recibe el listado.

Pregunta: ¿qué representa `data-producto-id`? Un atributo HTML que contiene el identificador del objeto actual. No es una ruta ni una variable global; sirve para identificar el producto en el documento generado.

Activar el Partial Tag Helper en el listado.

En `Views/Productos/Index.cshtml` busca el comentario `PASO 1.5.4`:

```
@* PASO 1.5.4 (inicio)
<section id="tarjetas-catalogo" aria-label="Tarjetas reutilizables de productos">
    <h2>Vista de tarjetas</h2>
    @foreach (var producto in Model)
    {
        <partial name="_TarjetaProducto" model="producto" />
    }
</section>
PASO 1.5.4 (fin) *@
```

Retira las dos líneas delimitadoras del comentario Razor. Conservar el `foreach` es fundamental: cada elemento de la colección se pasa por separado a la parcial.

La directiva de importación `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers` ya está presente en el proyecto. No necesitas instalar ningún paquete ni copiar manualmente el contenido de la parcial en el listado.

Comprobar las tres tarjetas por su identificador.

Compila e inicia. Abre `/Productos`. Deben aparecer el listado clásico y una nueva sección titulada **Vista de tarjetas**.

Comprueba que existen tres tarjetas y que sus atributos `data-producto-id` tienen los valores 1, 2 y 3. Revisa también que cada tarjeta muestra el nombre y el precio correctos.

No basta con comprobar que aparecen tres elementos visualmente parecidos. Cada tarjeta debe recibir un objeto diferente. Si las tres mostraran Cuaderno, por ejemplo, habría una incidencia en el dato que se pasa a la parcial o en la expresión que se representa.

Observar el HTML resultante.

En F12 → Red, inspecciona el documento principal y busca `class="tarjeta-producto"`. Debe aparecer tres veces.

No deberías encontrar tres nuevas solicitudes HTTP a `_TarjetaProducto.cshtml`. La parcial no es un recurso servido al navegador por separado: Razor la representa dentro de la respuesta de `/Productos`.

Paso 1.5.5 — Reutilizar la misma parcial en la vista de detalle

Detener la aplicación y abrir `Views/Productos/Detalle.cshtml`. Localizar el bloque:

```
@* PASO 1.5.5 (inicio)
<section id="tarjeta-detalle" aria-label="Ficha presentada como tarjeta">
    <h2>Tarjeta reutilizada en el detalle</h2>
    <partial name="_TarjetaProducto" model="Model" />
```

```
</section>
PASO 1.5.5 (fin) *@
```

Acción: retirar los dos delimitadores del comentario y dejar activo el elemento `<section>` completo.

Diferencia importante respecto al paso 1.5.4: en la vista del catálogo pasábamos `model="producto"` porque había un `foreach` sobre una colección. En la vista de detalle, el `Model` ya es un solo `Producto`, por lo que utilizamos `model="Model"`.

La parcial `_TarjetaProducto.cshtml` **no se copia** ni se duplica. Se encuentra en `Views/Shared` y el motor Razor la utiliza desde los dos archivos de vistas.

Compilar y ejecutar de nuevo. Visita `/Productos/Detalle/2`: bajo los datos de la ficha debe aparecer una tarjeta adicional de **Lápiz**, con su nombre, precio y enlace. Visita también `/Productos/Detalle/1`: debe aparecer una tarjeta de **Cuaderno**.

Regresar a `/Productos`: sus tres tarjetas continúan presentes y no han cambiado de estructura.

Conclusión del paso: el mismo archivo de parcial puede renderizarse muchas veces con modelos distintos, en páginas diferentes, sin duplicar marcado. Esta es la principal ventaja de la reutilización que se quiere enseñar en P1.5.

Profundización aplicada: Reutilizar la misma parcial en un contexto distinto.

Activar la parcial dentro del detalle.

Abre `Views/Productos/Detalle.cshtml` y localiza `PASO 1.5.5`:

```
@* PASO 1.5.5 (inicio)
<section id="tarjeta-detalle" aria-label="Ficha presentada como tarjeta">
    <h2>Tarjeta reutilizada en el detalle</h2>
    <partial name="_TarjetaProducto" model="Model" />
</section>
PASO 1.5.5 (fin) *@
```

Retira los delimitadores de comentario. Observa que el atributo `model` contiene ahora `Model`, no una variable `producto` creada por un bucle. La vista de detalle recibe un único `Producto`.

Comparar la cantidad de parciales representadas.

Visita `/Productos`: deben seguir apareciendo tres tarjetas. Visita `/Productos/Detalle/2`: debe aparecer una única tarjeta correspondiente a Lápiz. Visita `/Productos/Detalle/1` y comprueba la tarjeta de Cuaderno.

En ambos contextos el marcado procede del mismo archivo `_TarjetaProducto.cshtml`. No hemos creado una copia en `Views/Productos` ni hemos añadido otra acción para representar tarjetas.

Explicar la reutilización en términos de mantenimiento.

Si en el futuro hubiera que cambiar el texto «Precio» por otro, ¿cuántos archivos de tarjeta habría que modificar? Solo `_TarjetaProducto.cshtml`, siempre que las dos páginas continúen utilizándola.

Si copiáramos el marcado en la lista y en el detalle, habría que mantener ambas versiones. La práctica pretende demostrar el beneficio de evitar esa duplicación.

Profundización aplicada: Taller final de P1.5 — Rastrear una misma tarjeta por dos páginas.

La finalidad de este taller es construir una explicación completa y verificable de la reutilización de vistas parciales, no añadir más archivos ni duplicar código. Partimos del proyecto del capítulo 1.5 con los pasos 1.5.2 a 1.5.5 activos y con el reto de etiqueta económica habilitado.

Elige un producto y localiza su origen.

Abre `Controllers/ProductosController.cs` y localiza la construcción del catálogo. Selecciona el producto con identificador 2. Debe corresponder a Lápiz y tener un precio inferior a tres unidades.

El tipo `Producto` se encuentra en `Models/Producto.cs` y define las propiedades del objeto. La acción `Index()` devuelve la colección que contiene ese producto; la acción `Detalle(int id)` puede seleccionar el mismo registro para representar una ficha individual.

Anota qué diferencia existe entre ambas acciones: la primera entrega varios productos a una vista de listado y la segunda entrega uno solo a la vista de detalle. La parcial no necesita conocer esa diferencia, porque recibe directamente un objeto `Producto`.

Sigue la tarjeta desde Index.cshtml.

En `Views/Productos/Index.cshtml` localiza el bucle activo:

```
@foreach (var producto in Model)
{
    <partial name="_TarjetaProducto" model="producto" />
}
```

Cuando `producto.Id` es 2, el objeto que se pasa a la parcial representa a Lápiz. Abre `Views/Shared/_TarjetaProducto.cshtml` y comprueba que utiliza `@Model.Id`, `@Model.Nombre` y `@Model.Precio` para producir el fragmento HTML.

En el navegador abre `/Productos`. Mediante F12 → Inspector localiza el elemento con `data-producto-id="2"`. Comprueba que muestra el nombre Lápiz, su precio y la etiqueta `Precio económico`, activada por la condición del reto.

Sigue la misma tarjeta desde Detalle.cshtml.

Abre `Views/Productos/Detalle.cshtml` y localiza:

```
<partial name="_TarjetaProducto" model="Model" />
```

Ahora `Model` no es una colección: la vista de detalle recibe un solo producto. Por eso se entrega directamente a la parcial.

Visita `/Productos/Detalle/2` e inspecciona el elemento con `data-producto-id="2"`. Debe mostrar los mismos datos y la misma etiqueta. El archivo que define el marcado de tarjeta sigue siendo uno solo.

Explica por qué esta coincidencia no se debe a haber copiado el HTML del catálogo en el detalle. Se debe a que las dos vistas representan la misma parcial con el mismo tipo de objeto.

Comprueba un producto que no cumple la condición.

Visita `/Productos/Detalle/1` y examina la tarjeta de Cuaderno. Tiene `data-producto-id="1"` y no debe contener la clase `etiqueta-economico`, porque su precio supera el umbral de la condición.

Una explicación adecuada del reto tiene que cubrir ambos resultados: presencia de la etiqueta en Lápiz y ausencia en Cuaderno. Si solo se comprueba el caso positivo, no se puede distinguir una condición correcta de un error que muestre la etiqueta en todas las tarjetas.

Evalúa qué archivo modificarías ante tres cambios.

Cambio A: «Todas las tarjetas deben mostrar otro texto para el precio». El lugar principal de presentación es `_TarjetaProducto.cshtml`, porque lo reutilizan ambas vistas.

Cambio B: «Solo la página de detalle debe presentar una nota adicional bajo el contenido». El proyecto ya ilustra esta posibilidad mediante `@section NotasPagina` en `Detalle.cshtml` y `RenderSectionAsync` en el layout.

Cambio C: «El precio de Lápiz cambia en los datos». El valor se encuentra en la fuente de datos preparada por el controlador. No debes escribir otro precio fijo en la parcial para simular que has cambiado el modelo.

Este análisis permite separar cambios de datos, cambios de contenido compartido y cambios de estructura de página.

Reto: aplicación de los pasos

Reto 1.5 — Destacar productos económicos reutilizando una única parcial.

Enunciado

El catálogo de prueba tiene estos precios:

Producto	Precio
Cuaderno	4,50 €
Lápiz	1,20 €
Carpeta	3,75 €

Se pide que las **tarjetas de productos con precio inferior a 3 €** muestren el texto **Precio económico**. La condición debe escribirse **solo en la vista parcial** y afectar automáticamente tanto a las tarjetas del listado como a la tarjeta de la ficha de detalle.

Resultado esperado: Lápiz muestra la etiqueta; Cuaderno y Carpeta no. No cambies `ProductosController.cs`, los modelos ni los archivos de las vistas que incluyen la parcial.

Código del reto incluido en el proyecto

Abrir `Views/Shared/_TarjetaProducto.cshtml` y localizar este fragmento incluido inicialmente como comentario Razor dentro del artículo:

```
@* RETO 1.5 (inicio)
@if (Model.Precio < 3m)
{
    <strong class="etiqueta-economico">Precio económico</strong>
}
RETO 1.5 (fin) *@
```

Acción 1: eliminar únicamente las líneas que contienen `@* RETO 1.5 (inicio)` y `RETO 1.5 (fin) *@`, conservando intacto el contenido intermedio. La solución activa es:

```
@if (Model.Precio < 3m)
{
    <strong class="etiqueta-economico">Precio económico</strong>
}
```

Explicación de cada parte: `@if` ejecuta una condición de presentación; `Model.Precio` consulta el precio del objeto actual que la vista parcial recibió; `3m` es un literal decimal de C#; `<strong>` genera un

elemento HTML solo cuando el precio satisface la condición. La clase `etiqueta-economico` ya dispone de un estilo básico en `wwwroot/css/site.css`.

Acción 2: no modificar ningún otro archivo. La tarjeta parcial se utiliza tanto dentro de las tres tarjetas del catálogo como dentro de la ficha de un producto. Por tanto, una única modificación debe observarse en ambas ubicaciones.

Acción 3: compilar e iniciar:

```
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5091
```

Comprobar primero `http://localhost:5091/Productos`: debe aparecer **una sola etiqueta Precio económico** en la tarjeta de Lápiz. Las tarjetas de Cuaderno y Carpeta no deben recibir esa etiqueta.

Después visitar `http://localhost:5091/Productos/Detalle/2`: la tarjeta reutilizada de Lápiz debe mostrar también la etiqueta. Visitar `http://localhost:5091/Productos/Detalle/1`: la ficha de Cuaderno no debe mostrarla.

Solución razonada y preguntas

¿Por qué solo aparece una etiqueta en el catálogo? Porque se recorre una colección de tres productos, pero solo Lápiz tiene `Precio < 3m`.

¿Por qué el detalle de Lápiz cambia sin editar `Detalle.cshtml`? Porque `Detalle.cshtml` representa la misma parcial, pasándole el objeto de Lápiz. La regla se evalúa dentro de la parcial y produce el mismo resultado en ambos lugares.

¿Qué sucedería si copiáramos la condición en cada vista, en lugar de tenerla en la parcial? Tendríamos dos sitios donde mantener una misma regla de presentación; sería fácil modificar uno y olvidar el otro.

¿Hemos calculado un descuento real? No. La etiqueta es una indicación visual de ejemplo. No cambia precios ni aplica una política comercial.

¿Dónde comprobar el código? En este Markdown y en `PROYECTO/1.5/CursoWeb.Mvc/Views/Shared/_TarjetaProducto.cshtml`. El proyecto ya contiene el código; el alumno no necesita inventarlo ni buscar una solución externa.

Verificación del reto

Desde la **raíz del repositorio**, con la instancia manual detenida, ejecutar:

```
bash M01_MVC/TESTS/1.5_e2e.sh
```

El escenario `reto` activa automáticamente la condición en una copia temporal junto con los pasos 1.5.2–1.5.5. Compila, inicia Kestrel y verifica que existe **una** etiqueta en el catálogo, **una** en `Detalle/2` y **ninguna** en `Detalle/1`. También vuelve a comprobar que las páginas y rutas anteriores siguen funcionando.

Reto de etiqueta económica con comprobaciones positivas y negativas.

El reto consiste en representar una etiqueta visual únicamente para los productos cuyo precio es inferior a 3 €. El código preparado se encuentra **dentro de la parcial**, no en el controlador:

```
@* RETO 1.5 (inicio)
@if (Model.Precio < 3m)
```

```
{
    <strong class="etiqueta-economico">Precio económico</strong>
}
RETO 1.5 (fin) *@
```

Retira las dos líneas delimitadoras del comentario. No cambies los precios del catálogo ni agregues condiciones duplicadas en `Index.cshtml` o `Detalle.cshtml`.

Comprobar el resultado en el listado.

Compila y ejecuta. Abre `/Productos` y revisa las tres tarjetas. Solo Lápiz debe mostrar **Precio económico**, ya que su precio es 1,20 €, mientras Cuaderno y Carpeta superan el umbral.

Comprueba que la etiqueta aparece **una sola vez** en el conjunto de tarjetas. Si aparece en todas, revisa la comparación `Model.Precio < 3m` y confirma que no has trasladado el bloque fuera de la parcial.

Comprobar el resultado en el detalle de Lápiz.

Abre `/Productos/Detalle/2`. La tarjeta de Lápiz debe mostrar la misma etiqueta, aunque no hayas modificado la vista de detalle. La condición se evalúa dentro de la parcial al recibir el objeto Producto.

Este resultado demuestra la reutilización real: el mismo archivo aporta una regla de presentación coherente a dos vistas diferentes.

Comprobar los casos que no cumplen el criterio.

Abre `/Productos/Detalle/1` y observa la tarjeta de Cuaderno. No debe aparecer el texto «Precio económico». Después visita `/Productos/Detalle/3` y comprueba lo mismo para Carpeta.

Un test que solo comprobara la ficha de Lápiz sería insuficiente: una condición incorrecta podría generar el texto para todos los productos. Verificar también los casos negativos confirma que se está aplicando el criterio esperado.

Explicar por qué no es una regla comercial.

La condición de precio es una decisión visual de este laboratorio. No modifica el importe del producto, no aplica un descuento ni afecta a la persistencia. El propósito es demostrar que una regla simple de representación puede mantenerse en un solo archivo reutilizable.

En una aplicación real, las reglas que afectan a operaciones económicas o permisos de compra deben tratarse en las capas adecuadas. No deben depender exclusivamente de un texto mostrado por Razor.

Comprobaciones funcionales y casos de diagnóstico

Tabla de comprobación de la práctica.

Estado	Código activado	Qué debe poder comprobarse
Base	Ningún bloque nuevo	Listado y detalle de P1.4 funcionales
Condición Razor	1.5.2	Mensaje «El catálogo contiene al menos tres productos»
Sección del layout	1.5.2 + ambos bloques 1.5.3	Nota adicional en ficha; listado y portada funcionan sin sección
Tarjetas en listado	Anteriores + 1.5.4	Tres tarjetas procedentes de `_TarjetaProducto.cshtml`
Tarjeta en detalle	Anteriores + 1.5.5	Una tarjeta de producto en cada ficha

Estado	Código activado	Qué debe poder comprobarse
Reto	Todo lo anterior + RETO 1.5	Lápiz se etiqueta como económico en catálogo y detalle; los otros productos no

En cada etapa se debe confirmar al menos una vez que el código compila y el HTML esperado llega al navegador. Las pruebas automatizadas reconstruyen las seis etapas completas con una aplicación HTTP real.

Recorrido de regresión de todo M01.

Una vez activados todos los pasos y el reto, comprueba estas direcciones:

```
http://localhost:5091/
http://localhost:5091/Productos
http://localhost:5091/Productos/Detalle/1
http://localhost:5091/Productos/Detalle/2
http://localhost:5091/catalogo/ficha/2
http://localhost:5091/css/catalogo.css
```

Cada una representa una parte del módulo. La portada confirma la continuidad de 1.1; el listado utiliza los datos del modelo y la vista de 1.2; el CSS procede de la estructura de 1.3; las fichas usan las rutas y acciones de 1.4; y las tarjetas y el layout muestran la composición Razor de 1.5.

Al revisar `/Productos`, comprueba el contador de tres productos, el mensaje de catálogo completo, los enlaces de detalle y las tres tarjetas. Al revisar `Detalle/2`, comprueba el identificador, nombre y precio de Lápiz, la nota del layout y la tarjeta con la etiqueta económica.

Después solicita `Detalle/1` y confirma que la etiqueta no aparece en la tarjeta de Cuaderno. Esta última comprobación evita dar por finalizado el módulo tras observar solamente un caso positivo.

Comparar la ejecución manual con el script de pruebas.

Detén el servidor y ejecuta desde la raíz del repositorio:

```
bash M01_MVC/TESTS/1.5_e2e.sh
```

El script construye copias temporales de las variantes y comprueba las respuestas HTTP. Si supera sus escenarios, sabemos que el código preparado permite activar los pasos sin introducir errores detectados por esas pruebas.

Sin embargo, el alumno todavía debe explicar qué se ejecuta dentro del servidor, qué HTML recibe el navegador y por qué una vista parcial puede utilizarse en varios lugares. Los tests automáticos son una ayuda para la reproducibilidad, no sustituyen la comprensión del flujo.

Comprobación final por escenarios sin modificar la fuente.

Con la aplicación todavía ejecutándose, completa la tabla de resultados observados:

Dirección	Productos mostrados en tarjetas	Nota opcional del layout	Etiqueta económica
`/Productos`	Tres	No definida por esa vista	Solo en Lápiz
`/Productos/Detalle/2`	Una, Lápiz	Sí	Sí
`/Productos/Detalle/1`	Una, Cuaderno	Sí	No

Dirección	Productos mostrados en tarjetas	Nota opcional del layout	Etiqueta económica
`/Productos/Detalle/3`	Una, Carpeta	Sí	No

No debes modificar archivos para conseguir cada resultado: los cuatro casos utilizan el mismo estado final del proyecto. Esta es una prueba de que el comportamiento depende del modelo entregado a las vistas, no de crear una copia distinta de la plantilla para cada producto.

Detén el servidor y ejecuta la batería de P1.5 desde la raíz del repositorio:

```
bash M01_MVC/TESTS/1.5_e2e.sh
```

Al finalizar, explica qué estados reconstruye el script mediante copias temporales y por qué no modifica el proyecto inicial del alumno. Su objetivo es que los pasos activables y el reto permanezcan reproducibles incluso cuando se regresa a una fase anterior de la práctica.

Conclusión del taller

La competencia que debes demostrar no consiste solamente en saber escribir `<partial>`. Debes poder describir **cómo se selecciona la vista, qué objeto se entrega, en qué lugar se representa el fragmento y qué resultados positivos y negativos deben observarse**.

Si consigues seguir Lápiz desde su definición en el controlador hasta la tarjeta del catálogo y la ficha individual sin inventar archivos adicionales, has comprobado la principal ventaja de una vista parcial tipada: reutilizar representación coherente con un contrato de datos claro.

Resolución de incidencias

Problemas frecuentes y soluciones.

La vista no encuentra `_TarjetaProducto`. Comprobar que el archivo está en `Views/Shared/_TarjetaProducto.cshtml`, que se escribe su nombre de forma correcta en el `<partial>` y que se está ejecutando el proyecto de 1.5.

Aparece literalmente una etiqueta `<partial>` en el HTML sin las tarjetas. Revisar que `Views/_ViewImports.cshtml` conserva `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`. El Tag Helper necesita este registro.

Al activar el paso 1.5.3 se obtiene un error por secciones no representadas. Es necesario activar ambos bloques del mismo paso: `RenderSectionAsync` en el layout y `@section` en la vista de detalle. No hay que activar únicamente la mitad correspondiente a una vista.

La portada o el listado falla después de añadir la sección. Comprobar que en el layout se escribió `required: false`. Una sección obligatoria exigiría que todas las vistas del sitio la definieran.

El listado dice que no hay modelo. En el proyecto de 1.5 el controlador `Index` ya envía una colección. Verificar que se está ejecutando la carpeta correcta y no un proyecto anterior.

La vista parcial se genera tres veces dentro del detalle. Revisar que en `Detalle.cshtml` se usa un único `<partial model="Model" />` y no un bucle, porque `Model` ya es un objeto individual.

No aparece la etiqueta del reto. Revisar que se han quitado ambas líneas del comentario Razor del bloque `RETO 1.5`, que está activo el paso 1.5.4 o 1.5.5 que incluye la parcial y que se ha reconstruido la aplicación.

La etiqueta también aparece en Cuaderno. Comprobar que la comparación es estrictamente `Model.Precio < 3m` y que no se cambió el precio del producto.

Se intenta colocar el contenido de `@section` dentro de la parcial. Las secciones están asociadas a vistas completas y sus layouts. El ejercicio define `NotasPagina` en `Detalle.cshtml`, no en `_TarjetaProducto.cshtml`.

La lista muestra caracteres escapados al inspeccionar el HTML. Razor puede codificar caracteres como entidades HTML. El navegador presenta el texto correcto; esto no significa que se haya modificado el nombre del producto.

Distinguir cinco tipos de fallo.

Fallo 1: aparece el mensaje de catálogo completo, pero no las tarjetas. Has activado 1.5.2, pero quizá no 1.5.4. Comprueba el bloque de parcial del listado antes de modificar datos del controlador.

Fallo 2: la ficha muestra la tarjeta pero no la nota del layout. Revisa ambos bloques de 1.5.3 y comprueba que el nombre `NotasPagina` coincide en la definición y la representación.

Fallo 3: el listado no muestra los nombres correctos en las tarjetas. Comprueba que `model="producto"` está dentro del `foreach` y que la parcial utiliza `@Model.Nombre`. No escribas nombres fijos dentro de las tarjetas.

Fallo 4: el Tag Helper se muestra literalmente como etiqueta HTML. Revisa `_ViewImports.cshtml` y la directiva de Tag Helpers de MVC. La etiqueta `<partial>` debe ser procesada por Razor en el servidor.

Fallo 5: la etiqueta económica aparece en Cuaderno. Revisa la condición y el umbral. El precio de Cuaderno es superior a 3 €, así que no debería mostrarla.

La técnica común consiste en observar qué parte del resultado es incorrecta y regresar al archivo responsable de esa parte, evitando reescribir simultáneamente el controlador, la vista y el layout.

Resultado y continuidad

Verificación final de M01 y preparación de M02.

Tras completar el reto, recorre el sitio: portada, listado con tres productos, ficha de Lápiz, ficha de Cuaderno, ruta por atributos y recurso CSS. En el listado comprueba la condición de tres productos, las tarjetas y los enlaces; en el detalle comprueba la sección opcional del layout y la tarjeta reutilizada.

Cada funcionalidad debe poder asociarse a una pieza concreta: datos en el controlador, contrato `Producto` en el modelo, presentación en vistas Razor, estructura común en el layout y marcado repetido en la parcial.

El resultado de M01 es un catálogo consultable y navegable, todavía con datos en memoria. No hemos desarrollado todavía creación o edición de productos. En M02 añadiremos formularios, binding de GET/POST, validación, Tag Helpers de formularios y persistencia EF Core. Se mantiene así el alcance de las 30 horas acordadas.

Para continuar: conserva el proyecto 1.5 como referencia del catálogo MVC desarrollado. En M02 lo ampliaremos con operaciones de creación, edición y eliminación, manteniendo los conceptos de separación de responsabilidades, rutas y vistas que has practicado.

Síntesis integrada para finalizar la práctica.

Explica el recorrido completo desde que el navegador solicita `/Productos` hasta que se muestran las tres tarjetas. Debes mencionar la selección de la acción, la colección de modelos, la vista tipada, el `foreach`, el Partial Tag Helper, el archivo de parcial y el layout que proporciona la estructura compartida.

Después describe qué cambia cuando se solicita `/Productos/Detalle/2`. La acción recibe un identificador, entrega un único `Producto` y la vista de detalle utiliza la misma parcial con `model="Model"`. El layout puede añadir su sección opcional de notas.

Por último, justifica el resultado del reto: la condición de precio se ha escrito una vez en `_TarjetaProducto.cshtml` y produce una etiqueta en los contextos donde el modelo corresponde a Lápiz. No se ha duplicado el código en otras vistas ni se han cambiado los datos del controlador.

Conclusión práctica de M01: el alumno dispone de un catálogo MVC navegable, construido con rutas, modelos, controladores, vistas Razor, layout y una parcial reutilizable. La aplicación continúa utilizando datos en memoria. La creación y edición de productos, la validación de formularios y el almacenamiento con EF Core forman parte de M02, conforme a la progresión prevista del curso.